Ironwood Guide

The Zcash Orchard protocol and its Ironwood pool

m@rek.onl

Abstract

Assuming the *Math Guide*, the *Crypto Guide*, and the *Halo 2 Guide*, this volume of *The Zcash Arboretum* develops Zcash's Orchard shielded-payment protocol: the key hierarchy, notes and note commitments, the commitment tree and the procedure by which a wallet obtains a membership path, nullifiers, note encryption and trial decryption, value commitments and the RedPallas signature, the formal Action statement the SNARK proves, and the end-to-end security analysis that derives balance and nullifier uniqueness from knowledge soundness, and privacy from zero knowledge. NU6.3 splits the protocol's value across two pools — the legacy Orchard pool, sealed to spend-only, and the current Ironwood pool — and the volume develops the split, the quantum-recoverable Ironwood note format, and the cross-address restriction, closing with the Zcash proof-system lineage.

Contents

1 Introduction

This is the *Ironwood Guide*, the volume of *The Zcash Arboretum* that develops the cryptography behind Zcash's Orchard protocol as deployed. It assumes three earlier volumes: the *Math Guide* (the underlying mathematics), the *Crypto Guide* (hash functions, commitments, signatures, key agreement, zero knowledge, and the remainder of the primitive toolbox), and the *Halo 2 Guide* (the general-purpose proof system). This volume assembles those tools into the Orchard protocol as carried by its current pool, *Ironwood*. ZIP-229 fixes the vocabulary up front: the *Orchard protocol* is the shared cryptographic construction — curves, Sinsemilla, the Action circuit, notes, commitments, nullifiers, keys, and note encryption — and it survives; the *Orchard pool* is the legacy value pool it ran from NU5, sealed to spend-only at NU6.3; the *Ironwood pool* is the current pool.

We develop the protocol from the problems it solves. After we fix the curve-level preliminaries and the Sinsemilla hash, we develop each of the following as the answer to one sub-problem: how a unit of value is represented (notes and note commitments); how an owner proves ownership of a real, unspent note without revealing which one (the commitment tree and a membership path); how the protocol prevents double-spending (nullifiers); how a sender privately delivers and a recipient detects a note (note encryption and trial decryption); how value is conserved without disclosure (value commitments and the binding signature); and how an owner authorises a spend (RedPallas). We then show the procedure by which a node verifies a payment without ever seeing the note, state the Action relation formally, trace the end-to-end security reductions, and close the volume with the Zcash proof-system lineage from Sprout through Sapling to Orchard, and within Orchard the three circuit versions ending at Ironwood.

2 The Orchard application: the relation proved by the SNARK

Halo 2 is a general-purpose proving system; it has no notion of money. The Orchard application converts it into a shielded-payment protocol by fixing one specific relation — the *Action statement* — that every transaction must prove. This section develops that relation from the problem it solves rather than from the symbols it uses. It begins from the question of what a private payment must accomplish, derives each cryptographic object as the answer to one sub-problem, then traces a complete payment from Alice to Bob end to end, and only then states the Action relation formally.

2.1 The problem and the four requirements on a shielded payment

A transparent ledger (Bitcoin, Zcash's transparent pool) records, for every unspent output, its owner and its denomination (the number of units it holds), and a payment publicly transfers an output from one owner to another. A *shielded* payment must achieve the same effects — the transfer of value, the prevention of double-spending, and the conservation of total supply — while disclosing none of them. A private payment system must accordingly satisfy four requirements simultaneously:

1. *Represent a unit of value* so that its owner, value, and very existence stay hidden, yet its owner can later prove possession of it. (Notes and *note commitments* solve this, §2.5.)

2. *Prove possession of a real, unspent note* without identifying which note. (The *commitment tree* together with a membership proof solves this, §2.6.)

3. *Prevent double-spending* without a public list of which notes an owner spent. (*Nullifiers* solve this, §2.7.)

4. *Prove that no one created value* without revealing any amount. (*Value commitments* and the *binding signature* solve this, §2.9.)

These four requirements constitute the core of the protocol, and they rest on shared machinery. To render the section self-contained, we fix the build order by dependency — nothing appears before we define it — and lay it out here as a map:

1. **Preliminaries** (§2.2): the curve-level primitives every later object invokes — the hash-to-curve GroupHash, the coordinate map Extract, the point encoding $\star$, and domain separators.
2. **Sinsemilla** (§2.3): the commitment/hash built on those primitives, which both the note commitment and the Merkle tree use.
3. **Keys** (§2.4): the key hierarchy, from which the protocol derives addresses, commitments, and nullifiers.
4. **The first three requirements in turn**: notes (§2.5), the commitment tree (§2.6), and nullifiers (§2.7).
5. **Transmission** (§2.8): how the sender delivers the new note in-band and the recipient detects it.
6. **The fourth requirement**: value commitments (§2.9); together with *RedPallas* (§2.10), the signature that authorises a spend.
7. **Assembly** (§2.11): a complete Alice-to-Bob payment, end to end.
8. **The Action statement** (§2.14): the formal relation, the conjunction of in-circuit checks that makes all four hold at once.

Each later object depends only on earlier ones; the dependency root is the preliminaries below.

2.2 Preliminaries: hashing and encoding on the curve

The protocol builds every Orchard object below from a small set of curve-level primitives. We define them once, here, before anything uses them: a hash-to-curve GroupHash, the coordinate map Extract, the point encoding $\star$, and the domain separators that keep distinct uses independent. All operate on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$.

The coordinate map Extract**.** $\mathsf{Extract} : \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ maps a curve point to its x-coordinate, $\mathsf{Extract}(x, y) := x$ (with $\mathsf{Extract}(\mathcal{O}) := 0$ by convention). It appears wherever a later object must be a *field element* rather than a curve point — because the Merkle tree, the nullifier set, and the proof's public inputs are all field elements. It is not injective ($\pm(x, y)$ share an x), which is harmless here: the protocol only ever requires the x-coordinate as a binding digest, never to recover the point.

Point encoding: the $\star$ superscript. $P^\star \in \{0,1\}^{256}$ denotes the canonical bit-encoding of a point $P \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$: its x-coordinate (an element of $\mathbb{F}_{p_{\mathsf{Pallas}}}$, which fits in 255 bits, since $p_{\mathsf{Pallas}} < 2^{255}$) packed into a 256-bit string together with one sign bit, placed in the otherwise-unused top bit, recording which of the two square roots $\pm y$ is meant. Unlike Extract, this encoding is injective — it determines P completely — and it is what we feed, as a bit-string, into a hash or commitment.

Integer encodings LE_n**.** $\mathrm{LE}_n(m) \in \{0,1\}^n$ denotes the n-bit little-endian encoding of an integer $m \in \{0, \ldots, 2^n - 1\}$: the bit string $b_0 b_1 \cdots b_{n-1}$ with $m = \sum_{i=0}^{n-1} b_i \, 2^i$ (a field element encodes via its integer representative). It renders integers and field elements as bit strings wherever they enter a hash or commitment: LE_{32} indexes the Sinsemilla generator table (§2.3), LE_{10} tags the Merkle-tree layers (§2.6), and LE_{64}, LE_{255} encode note values and field elements (§2.5).

The hash-to-curve GroupHash**.** $\mathsf{GroupHash}^{\mathbb{G}} : \{0,1\}^* \times \{0,1\}^* \to \mathbb{G}$ takes a pair (domain separator D, message M) and returns a point of $\mathbb{G}$. It is *deterministic* (the same input always yields the same point), *public* (anyone can evaluate it), and *efficiently computable*. It instantiates the IETF hash-to-curve standard: hash_to_field expands (D, M) — via expand_message_xmd over BLAKE2b-512, with D embedded in the domain-separation tag — into *two* base-field elements; each passes through the simplified SWU map onto a curve isogenous to $\mathbb{G}$; and the sum of the two resulting points is carried through the isogeny to $\mathbb{G}$ (the *Crypto Guide* develops this construction in full). For everything that follows, we use only the properties that GroupHash is deterministic, public, and behaves like a random choice of point.

That last property is the purpose of the construction: because every stage is public and deterministic, the public strings D, M pin down the output $\mathsf{GroupHash}^{\mathbb{G}}(D, M)$, so no party can have chosen it to satisfy a hidden relation such as $P_2 = [s]P_1$ for a secret s. We call such points *nothing-up-my-sleeve* generators. For the security arguments we *model* GroupHash as a random oracle into $\mathbb{G}$ (the *Crypto Guide*): we treat its outputs as independent uniform points, so the family it produces has no efficiently-findable non-trivial discrete-log relation $\sum_i [a_i]P_i = \mathcal{O}$ — precisely the assumption the Sinsemilla binding and collision-resistance reductions invoke (§3.2). As with all random-oracle arguments this is a strong, standard, but unprovable modelling heuristic about the concrete hash, not a theorem.

Domain separators. A *domain separator* $D \in \{0,1\}^*$ is a fixed ASCII byte string naming the use site, fed to GroupHash so that logically distinct uses draw independent, unrelated generators (a collision found in one context cannot be transported to another). The strings are part of the protocol specification; the ones relevant here are

`z.cash : Orchard`	fixed generators $G^{\mathsf{SpendAuth}}$, G^{nf} (§2.4, §2.7),
`z.cash : Orchard-gd`	the diversified base $\mathsf{g_d}$ (§2.4),
`z.cash : Orchard-NoteCommit`	note commitments (§2.5),
`z.cash : Orchard-CommitIvk`	the ivk commitment (§2.4),
`z.cash : Orchard-MerkleCRH`	the Merkle-tree node hash (§2.6),
`z.cash : Orchard-cv`	the value-commitment bases (§2.9),
`z.cash : SinsemillaQ`, `z.cash : SinsemillaS`	the Sinsemilla Q point and S table (§2.3).

2.3 The commitment primitive: Sinsemilla

Two of the objects we build — the note commitment (§2.5) and the Merkle-tree node hash (§2.6) — require the same primitive: a commitment/hash that the spender must recompute *inside the zero-knowledge circuit*. In-circuit cost is therefore the binding constraint, and it is what Sinsemilla (Bowe and Hopwood) is designed to minimise. Its collision resistance reduces to DL on a prime-order curve, so it introduces no assumption beyond the DL hardness that Orchard's algebraic core already requires. It builds directly on the preliminaries of §2.2: its generators are GroupHash outputs, and its short form returns an Extracted coordinate.

Construction 2.1 (Sinsemilla)**.** Fix a prime-order group $\mathbb{G}$ (Orchard: $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$) and a chunk width k ($k = 10$ in Orchard). From the domain separator D, derive the $2^k + 1$ Sinsemilla generators:

$$Q(D) := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash : SinsemillaQ}, D) \in \mathbb{G},$$
$$P[m] := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash : SinsemillaS}, \mathrm{LE}_{32}(m)) \in \mathbb{G}, \qquad 0 \le m < 2^k.$$

The point $Q(D)$ is the domain-dependent starting accumulator; the 2^k points $P[0], \ldots, P[2^k - 1]$ form a fixed table indexed by a k-bit chunk value, shared across all Sinsemilla domains. For a

message M split into k-bit chunks $m_1, \ldots, m_\ell$ (so each $m_i \in \{0, \ldots, 2^k - 1\}$ indexes the table),

$$\mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell, \quad \mathsf{Acc}_0 := Q(D), \quad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \dotplus P[m_{i+1}]) \dotplus \mathsf{Acc}_i,$$

with $\dotplus$ *incomplete* short-Weierstrass addition. The output $\mathsf{Acc}_\ell \in \mathbb{G}$ is a curve point.

Theorem 2.2 (Sinsemilla collision resistance)**.** *For fixed-length inputs,* $\mathsf{Sinsemilla}_D$ *is collision-resistant assuming DL on* $\mathbb{G}$ *and modelling the generator derivation as a random oracle. (We prove this in §3.2.)*

Definition 2.3 (Sinsemilla commitment and short commitment)**.** The *Sinsemilla commitment* adds a hiding term to the hash:

$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D \,\|\, \texttt{-M}}(M) \dotplus [r]\, H_D \;\in\; \mathbb{G},$$

where $r \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment randomness (trapdoor), the hash runs in the suffixed domain $D \,\|\, \texttt{-M}$, and $H_D \in \mathbb{G}$ is a fixed blinding generator for the domain D, derived as the GroupHash output

$$H_D := \mathsf{GroupHash}^{\mathbb{G}}\big(D \,\|\, \texttt{-r},\ \varepsilon\big) \in \mathbb{G}$$

— the suffix `-r` appended to the domain string, with the empty message ε (a nothing-up-my-sleeve point, §2.2). Because H_D is a GroupHash output in its own sub-domain, no party knows a discrete-log relation between it and the generators $Q(D \,\|\, \texttt{-M}), P[m]$ that the hash uses; this independence is exactly what keeps the commitment binding — a party knowing such a relation could trade the blinding term against the message and open one commitment to two distinct messages — while hiding needs no assumption at all: for uniform r the term $[r]H_D$ is uniform on $\mathbb{G}$ and masks the hash completely. This is the same division of roles the independent base H supports in a Pedersen commitment (the *Crypto Guide*). The *short commitment* returns only its x-coordinate, a base-field element rather than a curve point:

$$\mathsf{ShortCommit}_r(D, M) := \mathsf{Extract}\big(\mathsf{SinsemillaCommit}_r(D, M)\big) \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}},$$

with Extract the coordinate map of §2.2. The unkeyed *short hash* omits the blinding term altogether:

$$\mathsf{ShortHash}_D(M) := \mathsf{Extract}\big(\mathsf{Sinsemilla}_D(M)\big) \;\in\; \mathbb{F}_{p_{\mathsf{Pallas}}},$$

the unblinded analogue of ShortCommit — no randomness argument, collision-resistant (Theorem 2.2) but not hiding.

The commitment is perfectly hiding and computationally binding — the two properties §2.5 requires of NoteCommit. Indeed the note commitment $\mathsf{NoteCommit}_{\mathsf{rcm}}(\cdot)$ is exactly $\mathsf{SinsemillaCommit}_{\mathsf{rcm}}$ under the domain `z.cash : Orchard-NoteCommit`. The short forms appear wherever the consumer requires a field element instead of a point: ShortHash in the Merkle-tree node hash (§2.6), ShortCommit in the incoming viewing key ivk (§2.4). The latter use enters through a named instance that fixes the domain and the message encoding:

$$\mathsf{Commit}^{\mathsf{ivk}}_{r_{\mathsf{ivk}}}(x, y) := \mathsf{ShortCommit}_{r_{\mathsf{ivk}}}\big(\texttt{z.cash : Orchard-CommitIvk},\ \mathrm{LE}_{255}(x) \,\|\, \mathrm{LE}_{255}(y)\big),$$

taking two field elements and concatenating their 255-bit little-endian encodings into the 510-bit message. Its output lies in $\{1, \ldots, p_{\mathsf{Pallas}} - 1\}$ (the zero output cannot occur for a valid key, so ivk is a usable scalar).

In-circuit cost. Each round consumes a whole k-bit chunk via one table lookup of $P[m]$ and two incomplete additions, so a 510-bit message costs 51 rounds rather than 510 bit-operations. In a PLONK-ish circuit with lookups (the *Halo 2 Guide*) this is dramatically cheaper than Sapling's bit-by-bit Pedersen hash — which is the reason the protocol changed hash between generations.

2.4 Keys: capabilities for spending and for viewing

We build the Orchard machinery bottom-up, beginning with the keys, because everything later — addresses, note commitments, nullifiers — derives from them. A practical requirement shapes the key hierarchy: different parties should be able to perform different operations. The owner can spend. An auditor should be able to *see* incoming and outgoing payments without being able to spend. A point-of-sale terminal should be able to *detect* incoming payments without being able to see outgoing ones. Each of these is a distinct capability, and the key tree makes each capability a separate derived key — the owner can hand out a lower-level key without surrendering the powers above it.

Definition 2.4 (Orchard key hierarchy)**.** The root secret is a 32-byte *spending key* sk, chosen uniformly at random when the owner creates the account (in a deployed wallet it typically derives from a seed phrase via ZIP-32 hierarchical-deterministic derivation, but for present purposes it is simply a uniform 256-bit string, $\mathsf{sk} \in \{0,1\}^{256}$). Every other key is a deterministic function of sk.

Expansion function. The expansion uses BLAKE2b-512, a conventional (non-arithmetisation-friendly) cryptographic hash with a 512-bit (64-byte) digest; we write PRF-style expressions with two arguments, the first a key and the second a message. A *pseudorandom function* (PRF) is a keyed family $\{f_K\}_K$ whose outputs are, for a secret uniform K, computationally indistinguishable from those of a truly random function; here the protocol obtains one not from BLAKE2b's native keyed mode but as *personalised, unkeyed* BLAKE2b over the concatenation $\mathsf{sk} \,\|\, t$, a PRF of t whenever sk is secret.

Accordingly, write $\mathsf{PRFexpand} : \{0,1\}^{256} \times \{0,1\}^{*} \to \{0,1\}^{512}$ for the PRF instantiated as $\mathsf{PRFexpand}(\mathsf{sk}, t) := \text{BLAKE2b-512}(\texttt{``Zcash\_ExpandSeed''}, \mathsf{sk} \,\|\, t)$ — unkeyed BLAKE2b-512 with the fixed 16-byte personalisation string ``Zcash_ExpandSeed'' and input $\mathsf{sk} \,\|\, t$, where the secret sk acts as the PRF key and the leading bytes of t are a domain-separation tag identifying which sub-secret it produces. (We write the key as a subscript, $\mathsf{PRFexpand}_{\mathsf{sk}}(t)$, when convenient.) We reduce its 512-bit output into the required field or scalar set. From sk derive the three spend-side secrets

$$\begin{aligned} \mathsf{ask} &:= \text{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\texttt{0x06})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \mathsf{nk} &:= \text{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\texttt{0x07})\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}}, \\ \mathsf{r}_{\mathsf{ivk}} &:= \text{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{sk}}(\texttt{0x08})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \end{aligned}$$

so that the *spend authorising key* ask and the *CommitIvk randomness* $\mathsf{r}_{\mathsf{ivk}}$ are scalars in $\mathbb{F}_{p_{\mathsf{Vesta}}}$ (the scalar field of Pallas, i.e. the integers mod the Pallas group order p_{Vesta}), while the *nullifier key* nk is a base-field element in $\mathbb{F}_{p_{\mathsf{Pallas}}}$. The two reduction maps read the 512-bit PRF output as an integer and reduce it modulo the relevant prime:

$$\begin{aligned} \text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Vesta}} \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \\ \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\mathsf{Pallas}} \in \mathbb{F}_{p_{\mathsf{Pallas}}}, \end{aligned}$$

where LEOS2IP_{512} is the little-endian octet-string-to-integer map sending a 64-byte string to the integer $\sum_{i=0}^{63} x_i \, 256^i \in \{0, \ldots, 2^{512} - 1\}$. ToScalar lands in $\mathbb{F}_{p_{\mathsf{Vesta}}}$, ToBase in $\mathbb{F}_{p_{\mathsf{Pallas}}}$; the two targets differ only because the protocol uses nk as a field element (it feeds it to the nullifier PRF, §2.7) whereas it uses $\mathsf{ask}, \mathsf{r}_{\mathsf{ivk}}$ as scalars multiplying group elements. Because the input is 512 bits wide while each modulus is just over 2^{254} (of bit length 255), the reduction is statistically indistinguish-

able from uniform on the target set (the modular bias is $O(2^{254}/2^{512}) = O(2^{-258})$, negligible). The distinct tag bytes 0x06, 0x07, 0x08 make the three outputs independent.

Spend validating key. Let $G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ be a fixed, publicly-known generator of the Pallas group, obtained by $G^{\mathsf{SpendAuth}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard}, \texttt{G})$ (a nothing-up-my-sleeve point, §2.2). The *spend validating key* is then the public point

$$\mathsf{ak} := [\mathsf{ask}]\, G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

Key generation canonicalises its sign: if $[\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$ has odd y-coordinate, ask is replaced by $-\mathsf{ask}$, so that ak always has even y and the x-coordinate $\mathsf{Extract}(\mathsf{ak})$ alone determines the point. The *full viewing key* bundles the three quantities needed to recognise and decrypt one's own notes, $\mathsf{fvk} := (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathbb{F}_{p_{\mathsf{Vesta}}}$. From it derive

$$\begin{aligned}
\mathsf{ivk} &:= \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}\big(\mathsf{Extract}(\mathsf{ak}),\ \mathsf{nk}\big) \in \{1, \ldots, p_{\mathsf{Pallas}} - 1\} \subset \mathbb{F}_{p_{\mathsf{Pallas}}},\\
K &:= \mathrm{LE}_{256}(\mathsf{r_{ivk}}) \in \{0,1\}^{256},\\
R &:= \mathsf{PRFexpand}\big(K,\ \texttt{0x82} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})\big) \in \{0,1\}^{512},\\
(\mathsf{dk}, \mathsf{ovk}) &:= \big(R_{[0\,..\,256)},\ R_{[256\,..\,512)}\big) \in \{0,1\}^{256} \times \{0,1\}^{256},
\end{aligned}$$

where $\mathsf{Commit}^{\mathsf{ivk}}$ is the named ShortCommit instance of §2.3 and LE_{256} the 256-bit little-endian bit encoding of §2.2, giving K the type $\{0,1\}^{256}$ that PRFexpand requires of its key. This yields the *incoming* viewing key ivk (a base-field scalar), the *diversifier key* dk, and the *outgoing* viewing key ovk. The dk/ovk derivation is precisely the following: we split a *single* 512-bit PRFexpand output R, keyed by $\mathsf{r_{ivk}}$ over the input $\texttt{0x82} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk}$ (not over an empty message), into its two 256-bit halves — dk the first, ovk the second. The diversifier d below is *not* part of this split; it derives afterwards from dk. Here $\mathsf{Extract}(\mathsf{ak})$ is the x-coordinate of ak (§2.2) — which, after the sign canonicalisation above, determines ak — while $\mathrm{I2LEOSP}_{256}$ denotes the 32-byte little-endian encoding of a field element or point. A *diversified address* is, for any index $d_{\mathrm{plain}} \in \{0,1\}^{88}$,

$$\begin{aligned}
d &:= \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(d_{\mathrm{plain}}) \in \{0,1\}^{88},\\
\mathsf{g_d} &:= \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd}, d) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),\\
\mathsf{pk_d} &:= [\mathsf{ivk}]\, \mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),
\end{aligned}$$

where $\mathsf{FF1\text{-}AES256}_{\mathsf{dk}}$ produces the *diversifier* $d \in \{0,1\}^{88}$, the NIST FF1 format-preserving encryption mode (built on AES-256) keyed by dk: a keyed *permutation* of the 88-bit index space $\{0,1\}^{88}$, so that each d_{plain} maps to a distinct 88-bit diversifier d and different d_{plain} give unlinkable-looking addresses, all derived from the one key. The *address* is the pair $\mathsf{addr} := (d, \mathsf{pk_d}) \in \{0,1\}^{88} \times \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$.

Choosing the index d_{plain}. The receiving wallet chooses the index freely: *any* of the 2^{88} values yields a valid, spendable address for the same ivk. No derivation fixes it and no index is forbidden — any 88-bit string is a valid diversifier index. The wallet allocates indices on demand, conventionally sequentially from $d_{\mathrm{plain}} = 0$ (so index 0 is the *default diversifier*), assigning a fresh index per payee or per invoice to keep their addresses mutually unlinkable; a randomly drawn index would yield an equally valid address, but the specification states that diversifier indices SHOULD NOT be chosen at random: ZIP-32 specifies their usage in hierarchical-deterministic wallets, whose seed-only recovery of issued addresses depends on deterministic allocation. Every index works for a reason specific to Orchard: the hash-to-curve $\mathsf{GroupHash}^{\mathsf{Pallas}}$ of §2.2 is total — it returns a point for every input — so every index yields a well-defined $\mathsf{g_d}$; in the negligible-probability event that the two summed SWU outputs cancel to $\mathcal{O}$, the specification substitutes the fallback $\mathsf{GroupHash}(D, \texttt{""})$. This is a deliberate improvement over Sapling, where the diversifier hash failed on roughly half of all indices and the wallet had to increment d_{plain} until it found a valid one; in Orchard no such rejection loop exists.

> *Rationale for routing the index through AES.* The step from d_{plain} to d is a keyed *permutation* (format-preserving encryption) for three reasons that no simpler map satisfies together. (i) *Unlinkability:* wallets allocate indices in order $0, 1, 2, \ldots$, and consecutive diversifiers would reveal a shared owner and an issue count; encryption under dk scatters them pseudorandomly over $\{0,1\}^{88}$. (ii) *Bijection:* a permutation never collides two indices onto one d and is invertible — with dk the wallet recovers the index behind any of its addresses, storing no per-address secret; a plain hash is neither collision-free nor invertible. (iii) *Format preservation:* the address format fixes the diversifier at exactly 88 bits, and FF1 maps $\{0,1\}^{88} \to \{0,1\}^{88}$ bijectively, which a truncated hash cannot. This derivation runs *wallet-side, never in the circuit*, so a conventional fast cipher (AES-256 under NIST FF1) is appropriate; arithmetisation-friendliness is irrelevant here.

The structure is a deliberate capability ladder (Figure 1), each rung conferring strictly less power than the one above:

- sk / ask — *spend*. Whoever holds ask can authorise spends; it sits at the top and nothing below it can recover it.
- fvk (hence ivk, ovk) — *see everything*. Detects incoming notes (via ivk, which recovers $\mathsf{pk_d}$) and views outgoing notes (via ovk), but cannot spend, because deriving fvk from sk is one-way.
- ivk alone — *detect incoming*. Sufficient to recognise notes addressed to the holder and scan the chain, nothing more — the key suitable for an always-online wallet server.
- A diversified address $(d, \mathsf{pk_d})$ — *receive*. A public handle anyone can pay; one ivk generates 2^{88} mutually unlinkable addresses — one per diversifier index (Definition 2.4), practically inexhaustible — all spendable by the same key.

Of these, only the diversified address is *public*. The viewing keys fvk, ivk, ovk are *secret* key material — "viewing" names what they let the holder *do* (see transactions), not that one may publish them. Disclosing a viewing key confers the corresponding visibility on the recipient — every payment to and from the wallet for fvk, incoming payments only for ivk, outgoing only for ovk — so the owner shares one only deliberately (with an auditor, say) and otherwise guards it like any other secret. The ladder is useful precisely because these capabilities are *separable*: one can delegate "see incoming" (ivk) without delegating "see everything" (fvk), and neither confers the ability to spend.

Two questions of motivation that the bare formulas do not answer. *Why is* ivk *a commitment,* $\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$*, rather than a plain hash?* Because ivk must both bind *and* hide. It binds ak (spend authority) and nk (nullifier derivation) to one identity, and check A7 below proves the linkage in-circuit by recomputing the commitment from the witnessed ak, nk, $\mathsf{r_{ivk}}$ — but binding alone a collision-resistant hash would supply equally, and in-circuit recomputation works for any hash (check A3 recomputes the unblinded Sinsemilla MerkleCRH; Sapling even proved its BLAKE2s-based $\mathsf{CRH}^{\mathsf{ivk}}$ in-circuit). What the trapdoor $\mathsf{r_{ivk}}$ adds is *hiding*: the blinded ShortCommit is perfectly hiding, so ivk — and hence every public $\mathsf{pk_d} = [\mathsf{ivk}]\mathsf{g_d}$ — reveals nothing about ak and nk, whereas the unblinded Sinsemilla hash is only collision-resistant (Theorem 2.2) with no one-wayness claim. Hiding is what keeps the IN/OUT tier strictly below FULL VIEW: an ivk holder, such as an always-online wallet server, cannot extract nk or ak. *Why is* nk *separate from* ask*?* So that the owner can delegate the capability to compute nullifiers (which a viewing key needs to determine which of the holder's notes are already spent) *without* conferring the ability to spend — the viewing key holds nk but never ask.

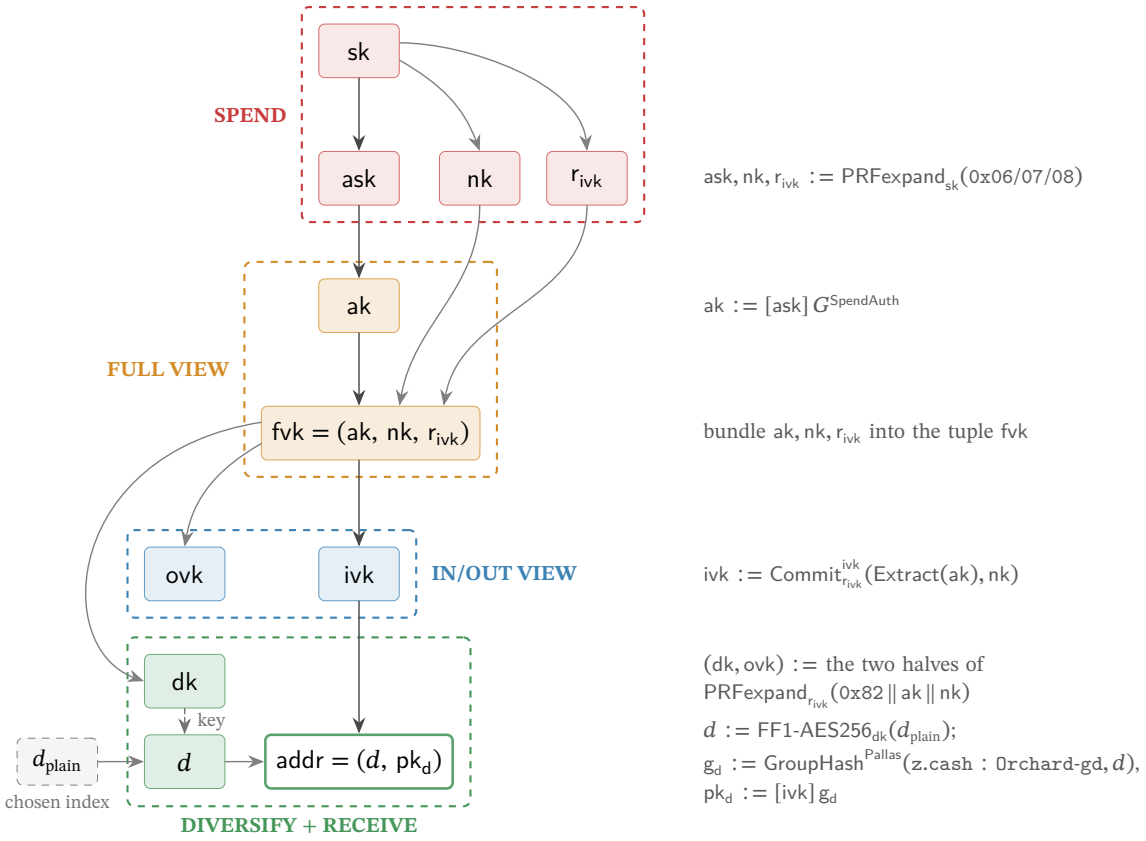

Figure 1: The Orchard key hierarchy as a capability ladder, read top to bottom. Each arrow is a derivation; the right-hand gutter shows the operation that produces the child from its parent(s). Not every edge is one-way: the bundling edges (ak, nk, r_{ivk} into the tuple fvk; d into the address) invert by projection, and the FF1 edge is a keyed permutation the dk holder inverts — the tiers are separated by the one-way steps sk → (ask, nk, r_{ivk}), ask → ak, fvk → (ivk, dk, ovk), and ivk → pk_d. The dk and ovk keys are the two halves of one $\mathsf{PRFexpand}_{r_{ivk}}$ output (tag 0x82, input ak||nk); they play different roles and land in different tiers — ovk is the *outgoing* viewing key, grouped with the *incoming* viewing key ivk as the IN/OUT viewing tier, whereas dk is the *diversifier* key, which sits in the address layer because it generates diversified addresses. Concretely d is the format-preserving encryption FF1-AES256 of a chosen index d_{plain} under the key dk (the dashed edge marks dk entering as the cipher key), and $\mathsf{pk}_d = [\mathsf{ivk}]\,\mathsf{g}_d$, so the address binds both keys. *Every node is secret key material except the chosen index d_{plain}, the diversifier d, and the diversified address (unshaded) — the address (d, pk_d), and with it d, is the only value the protocol ever publishes* (shading marks tier membership, not secrecy) — in particular the viewing keys fvk, ivk, ovk are *secret:* "viewing" names what they let the holder do, not that the protocol publishes them. Each dashed tier holds a strictly weaker capability than the one above, so a holder may receive a lower-tier key without gaining the powers above it. Drawn to match Definition 2.4.

2.5 Representation of a note: notes and note commitments

The first requirement is to represent a note such that it stays invisible on-chain yet provable by its owner. The protocol therefore never places the note on the chain at all; it publishes only a hiding commitment to it.

Definition 2.5 (Orchard note)**.** An Orchard *note* — the private object representing one unit of value — is a tuple

$$\mathbf{n} := (\mathsf{addr}, v, \rho, \psi, \mathsf{rcm}),$$

where $\mathsf{addr} := (d, \mathsf{pk_d})$ is the recipient's diversified address; $v \in \{0, \ldots, 2^{64} - 1\}$ is the value in zatoshi; $\rho \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a per-note uniqueness seed; $\psi \in \mathbb{F}_{p_{\mathsf{Pallas}}}$ is randomness the sender fixes through the note seed rseed, from which it is derived deterministically (§2.8); and $\mathsf{rcm} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ is the commitment trapdoor.

The protocol never publishes the note itself. What it records on-chain is its *note commitment*

$$\mathsf{cm} := \mathsf{NoteCommit}_{\mathsf{rcm}}\big(\mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi)\big) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

a commitment that is *perfectly hiding* (the chain learns nothing about $\mathsf{addr}, v, \rho, \psi$ from cm) and *computationally binding* (the sender cannot later claim the commitment was to a different note). These are precisely the two properties of a commitment scheme from the *Crypto Guide*; here the scheme is Sinsemilla (§2.3), chosen because it is inexpensive to recompute *inside the circuit*, which the spender must do. The chain stores $\mathsf{cmx} := \mathsf{Extract}(\mathsf{cm}) \in \mathbb{F}_{p_{\mathsf{Pallas}}}$, the x-coordinate of the commitment point, because the Merkle tree consumes a field element.

The three address-related quantities appearing in NoteCommit are the key material that §2.4 defines: the diversifier $d \in \{0,1\}^{88}$, the diversified base $\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ (the per-address generator $\mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd}, d)$), and the transmission key $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ that identifies the recipient; the address is $\mathsf{addr} = (d, \mathsf{pk_d})$. Recall the $\star$-encoding from §2.2: $\mathsf{g_d}^{\star}, \mathsf{pk_d}^{\star} \in \{0,1\}^{256}$. The NoteCommit input is the bit-string concatenation ($\|$) of these point encodings with the little-endian integer encodings $\mathrm{LE}_{64}(v) \in \{0,1\}^{64}$ and $\mathrm{LE}_{255}(\rho), \mathrm{LE}_{255}(\psi) \in \{0,1\}^{255}$.

The choice of fields is as follows. addr and v are the note's content. ρ and ψ exist solely to render the eventual nullifier unique and unpredictable; their role becomes apparent only in §2.7: a note carries randomness whose purpose is downstream.

2.6 Proof of ownership of *some* valid note: the commitment tree

The second requirement is to prove the spent note is valid without revealing which note. When Alice spends, she must convince the network that the note she is spending is a valid note that someone actually created; otherwise she could spend a note she invented. On a transparent chain this is trivial: the output is present in the set of unspent transaction outputs (the UTXO set) that every node maintains. But Alice must prove it *without revealing which note*, because revealing the note would deanonymise the payment.

The network maintains one append-only Merkle tree whose leaves are *every note commitment ever published*, in creation order. Alice proves *membership*: a Merkle authentication path from her leaf cmx to a root the network trusts. Inside the circuit the path is a private witness (check A3), so the verifier learns that her note is one of the leaves but not which one; the anonymity set is the entire tree.

Definition 2.6 (Orchard commitment tree)**.** The Orchard *note commitment tree* is an append-only Merkle tree of fixed depth 32 (so up to 2^{32} notes). Leaves are cmx values in creation order. The

parent of a left child ℓ and right child r at layer ℓ' is

$$\mathsf{MerkleCRH}^{\mathsf{Orchard}}_{\ell'}(\ell, r) := \mathsf{ShortHash}_{\texttt{z.cash:Orchard-MerkleCRH}}(\mathrm{LE}_{10}(\ell') \,\|\, \mathrm{LE}_{255}(\ell) \,\|\, \mathrm{LE}_{255}(r)) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$

a layer-tagged Sinsemilla short hash (Definition 2.3) of the $10 + 255 + 255 = 520$-bit message — 52 ten-bit chunks, the same LE-encoded input convention as $\mathsf{Commit}^{\mathsf{ivk}}$ and NoteCommit (leaves cmx, internal nodes, and the root all live in $\mathbb{F}_{p_{\mathsf{Pallas}}}$). Appending a block's note commitments yields a new root: the root of the tree after block h is the *anchor* of block h (a single field element), so every main-chain block defines exactly one anchor. A spend proves membership against the anchor of any main-chain block, and all Actions of one bundle prove against a single shared anchor (§2.13).

Every node is a single base-field element by necessity, not convention. The spender recomputes this hash layer by layer *inside* the Action circuit to prove membership (check A3 of §2.14), and that circuit is arithmetic over $\mathbb{F}_{p_{\mathsf{Pallas}}}$: every wire carries one such field element. A curve-point node would carry two coordinates and an on-curve constraint at each of the 32 layers; reducing each node to its x-coordinate halves the path data the circuit threads and removes that bookkeeping. This is precisely why MerkleCRH ends in Extract: Sinsemilla yields a point, and Extract returns it to the field, so that each layer's output is directly the next layer's input and the one hash composes all the way to the root. Dropping the y-coordinate is harmless here (§2.2): the tree uses each node only as a binding digest, never to recover a child point, and a collision in the x-coordinate still reduces to one in Sinsemilla (Theorem 2.2).

Two design details are significant. The tag carries the layer index ℓ' so that a path at one height cannot replay at another. And empty leaf positions take an "uncommitted" sentinel value 2: the choice is not arbitrary — at $x = 2$ the curve equation gives $y^2 = 2^3 + 5 = 13$, and 13 is a non-square modulo both Pasta primes (the *Math Guide* verifies this), so 2 is not the x-coordinate of any Pallas point, hence can never equal a valid cmx and can never be silently confused with a genuine commitment.

Derivation of the authentication path. Alice's commitment occupies a fixed *position* pos — its index in creation order — which her wallet records when it detects the note (§2.8). Fix an anchor of any block at or after the one that mined her note, and let n be the number of leaves in that block's tree. The authentication path of leaf pos against that anchor consists of one node per level: writing $\mathsf{pos} = b_{31} \cdots b_1 b_0$ in binary, the level-i ancestor of her leaf is a left child when $b_i = 0$ and a right child when $b_i = 1$, and the path's level-i entry is that ancestor's *sibling* in the tree of size n (Figure 2). Check A3 reverses the construction: starting from cmx it combines the running node with one path entry per level — placing the node on the side b_i dictates (left when $b_i = 0$, right when $b_i = 1$) and the sibling opposite — and asserts that the final value equals the public anchor; the bits b_i and the siblings stay inside the witness.

Each entry is one of two kinds. When $b_i = 1$ the sibling subtree lies to the *left* of her leaf: it spans commitments older than hers, so every one of its positions is filled — a *complete* subtree — and, because the tree appends only on the right, its root is final and identical at every anchor. When $b_i = 0$ the sibling subtree lies to her *right*, and its value depends on the anchor: its root hashes the commitments it holds at size n, with positions not yet filled taking the uncommitted sentinel 2; a sibling subtree still entirely empty at size n contributes a per-level constant, the sentinel folded up through the layer-tagged hash. The path is therefore a deterministic function of pos and the first n public leaves, and of nothing else; the only secret is which leaf is hers.

In practice the wallet neither rebuilds the path from the leaves on each spend nor retains the whole tree. As it scans, it appends every published cmx — its own and everyone else's — but keeps only what a future path needs: the nodes from each of its own notes up to the root, the complete-subtree roots flanking those paths, and one *checkpoint* per block; the rest it prunes (this is the design of librustzcash's shardtree). It marks its note's position the moment trial decryption succeeds (§2.8). A checkpoint stores merely the position of that block's last leaf, equivalently the leaf count n — the

size that fixes the block's anchor; the anchor itself is recomputed from the retained nodes on demand, never stored.

To produce the path for a spend against a chosen block, the wallet descends its retained tree to the marked leaf and, on the way back to the root, reads off each sibling subtree's root: a complete subtree contributes its stored root, a subtree lying wholly beyond the leaf count n contributes the per-level empty root, and the single subtree straddling position n is rehashed from its retained nodes. The leaf count enters only as this cutoff — every position $\geq n$ counts as empty — so one retained tree reproduces the path against any block it has checkpointed.

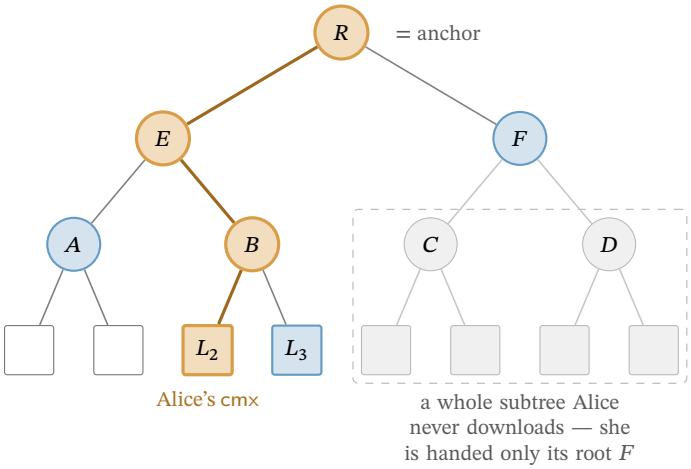

Figure 2: Computation of a Merkle authentication path (drawn at depth 3; Orchard's tree has depth 32). The leaves are the public note commitments cmx, in order. Alice's note sits at leaf L_2 (orange); to prove it is in the tree she must compute the authentication path of L_2, then recompute the bold route from L_2 up to the root R — the *anchor* — folding in, at each level, the sibling hash beside it: that path is L_3, A, F (blue). Efficiency is the only reason not to hash the whole tree: a far subtree enters her path solely through its single root (here F), so she never downloads its leaves (faded) — the GetSubtreeRoots shortcut that lets even a freshly restored wallet witness a years-old note.

Nothing above requires Alice to have been online: the inputs are public, so a freshly restored wallet rebuilds the same state by replaying the chain's note commitments. Nor need it touch all 2^{32} positions, and here the tree's structure earns its keep. Cut the tree at half its depth. Below the cut lie the *shards* — the height-16 subtrees, 2^{16} leaves apiece, up to 2^{16} of them — and above it a height-16 *cap* whose own leaves are the shard roots (librustzcash sets the shard height to half the tree depth, $32/2 = 16$). A leaf's 32 siblings divide along the same cut: the low 16 (levels 0 through 15) lie inside its own shard, and the high 16 (levels 16 through 31) are cap nodes, each the root of a subtree spanning whole shards. The two halves reach Alice by entirely different routes, and that asymmetry is the economy of the scheme.

Her own shard she must hash from leaves. Its 16 low siblings are the within-shard subtrees of the descent above — complete to her left, straddling or empty to her right — so she scans that shard's commitments block by block, her own and everyone else's, and folds them; this is the one subtree whose 2^{16} leaves she handles individually. Every *other* shard she takes as a single 32-byte root. The cap's leaves are exactly those roots, which a light server streams one per completed shard (GetSubtreeRoots); Alice folds them upward with the same layer-tagged MerkleCRH, one application per cap level — the upper half of the tree's 32 layers — and reads her high siblings off the result (Figure 2 is the toy-depth analogue: the faded subtree enters only through its root F). A shard that does not yet exist — the empty right of the cap — contributes the per-level empty constant rather than a download, and her own shard she holds from leaves whether or not it has completed, so she never needs its root from the server. Her level-16 sibling is thus a neighbouring shard's root, and each sibling above it a fold of $2, 4, 8, \ldots$ roots she already holds.

A concrete case fixes the shape. Let Alice's note be the *last* leaf of the newest shard k. Below the cut, all 16 low siblings are complete left subtrees tiling the shard's earlier $2^{16} - 1$ positions, so she ingests essentially the whole shard. Above the cut, the cap's right side is empty — no shard succeeds k, so those siblings are the empty constant — while its left side folds the roots of shards $0, \ldots, k-1$, each supplied by GetSubtreeRoots. Stitching the halves — her leaf folds up to the shard-k root, which folds up the cap to the anchor — gives the full 32-level path.

The 2^{16} throughout bounds the cap's *capacity* — 2^{32} leaves over 2^{16} per shard — not Alice's traffic. She fetches one root per shard that actually exists, $\lceil n/2^{16} \rceil$ of them: a few hundred at present tree sizes, and growing by one every 2^{16} outputs. The whole history above her shard thus arrives as a few kilobytes of roots in place of millions of leaves. She performs the fold *locally*, never asking a server for one particular note's path, which would reveal which note is hers; the finished path is the private witness of check A3, and only the anchor is public.

Anchor choice as the tree grows. By the time Alice's transaction reaches a block, the live tree has advanced past her chosen anchor. This costs nothing: appending leaves creates new roots and alters no old one, so the anchor still names one fixed historical tree, her path still folds to exactly that root, and check A3 asserts exactly that equality — collision resistance of the layer-tagged hash (§2.3) makes a path to that root for a non-member infeasible. Consensus accepts the anchor of *any* main-chain block from Orchard activation onward (NU5) — for the legacy Orchard-pool tree: the two pools keep separate, independent commitment trees and nullifier sets (ZIP-229), and an Ironwood Action instead proves against anchorIronwood over the Ironwood tree, empty at NU6.3 activation, whose anchors are valid from activation onward. In neither pool is there a sliding window, so proving is fully decoupled from broadcasting — Alice need not race the chain tip. The one recency caveat runs in the opposite direction: a reorganisation can strike the anchoring block from the main chain, invalidating the anchor and forcing her to rebuild the transaction; deeper anchors make the event rarer, but no depth rules it out — the often-cited 100-block figure is the specification's description (protocol § 3.3; the retired zcashd actually enforced 99), and the deployed zebra rolls back automatically to 1000 blocks, node policy in both validators rather than consensus (*Consensus Guide*, §"Reorg rails, maturity, and the finality floor"). Deployed wallets nonetheless anchor close to the tip, by deliberate choice rather than necessity. The anchor's tree is the anonymity set in which check A3 conceals the spent note — the proof reveals only that the note is *some* leaf committed by that anchor — so the most recent admissible anchor, naming the largest such tree, maximises that set, while an idiosyncratically old or deep anchor would instead fingerprint the spender. librustzcash accordingly selects the most recent block it has checkpointed that lies at least a fixed number of confirmations behind the tip (by default, per ZIP-315, 3 confirmations for notes the wallet produced itself and 10 for notes received from third parties), trading a sliver of reorg robustness for set size and uniformity. Consensus mandates none of this; it is wallet policy. Nor is an old anchor a soundness hole: membership in an older, smaller tree implies the note was committed even earlier, which is precisely what Alice must establish — the nullifier (§2.7), not the anchor, prevents a second spend.

2.7 Prevention of double-spending: nullifiers

The third requirement: to prevent Alice from spending the same note twice, without a public registry of spent notes (which would leak exactly what is being hidden). Each note yields a single deterministic tag — its *nullifier* — that appears only when its owner spends the note. The network maintains a set of all revealed nullifiers and rejects any repeat. The nullifier must satisfy two properties that pull in opposite directions: it must be *deterministic* (so the same note always yields the same tag, making a second spend detectable) yet *unlinkable* (so that publishing it reveals nothing about which note or address it came from).

Definition 2.7 (Orchard nullifier)**.** For a note **n** with commitment cm and the owner's nullifier key nk,

$$\mathsf{nf} := \mathsf{Extract}\big(\big[\,F_{\mathsf{nk}}(\rho)+\psi\,\big]G^{\mathsf{nf}}+\mathsf{cm}\big)\ \in\ \mathbb{F}_{p_{\mathsf{Pallas}}},$$

where $F_{\mathsf{nk}} : \mathbb{F}_{p_{\mathsf{Pallas}}} \to \mathbb{F}_{p_{\mathsf{Pallas}}}$ is a circuit-efficient PRF (Poseidon over the Pallas base field, the *Crypto Guide*), $G^{\mathsf{nf}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard, K}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ is a fixed nothing-up-my-sleeve Pallas generator (§2.2), the scalar $F_{\mathsf{nk}}(\rho)+\psi$ lives in $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (a valid Pallas scalar because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$), and Extract takes the x-coordinate. (We write G^{nf} for this base to distinguish it from the spend-auth generator $G^{\mathsf{SpendAuth}}$ of §2.4.)

We read the construction against the two requirements. Determinism: every ingredient $(\mathsf{nk}, \rho, \psi, \mathsf{cm})$ is fixed once the note exists, so nf is a fixed function of the note — spending it twice yields the same nf twice, and the network rejects the second. Unlinkability: the term $[F_{\mathsf{nk}}(\rho)+\psi]\,G^{\mathsf{nf}}$ masks the commitment with a value that appears random to anyone without nk, so no one can tie the published nf back to cm or to the address (this is where the note's ρ and ψ perform their function). The security properties:

- *No double-spend* (the Orchard Book calls this *balance*, since double-spending is how one would create value): each note has exactly one nullifier, and forging a colliding nullifier for a different note reduces to DL on Pallas.
- *Spend unlinkability*: without nk, the published nf is unlinkable to the note or address, under the Decisional Diffie–Hellman (DDH) assumption on Pallas — that $([a]G, [b]G, [ab]G)$ is computationally indistinguishable from $([a]G, [b]G, [c]G)$ for independent uniform a, b, c — together with the PRF assumption on F.
- *Forward consistency (Faerie resistance)*: a freshly created note takes as its uniqueness seed the nullifier of the note spent alongside it, $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$; since the network already guarantees nullifiers are unique, this forces every note's ρ to be globally unique, closing an attack from earlier designs in which forged notes could be made to share a nullifier.

2.8 Transmission and detection of a note: encryption, trial decryption, and synchronisation

The on-chain cmx conceals the note entirely, which leaves a question the commitment alone does not resolve: how does the recipient learn of the note with which they were paid? The sender must deliver the note to them, privately, over the same public chain. Orchard achieves this with *in-band secret distribution*: every Action carries, beside its commitment, a ciphertext only the recipient can open.

Encryption (sender side). Alice holds Bob's address $(d, \mathsf{pk_d})$, with $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and $\mathsf{g_d} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-gd}, d)$ (§2.4). She forms the *note plaintext* $\mathsf{np} = (\texttt{0x03}, d, v, \mathsf{rseed}, \mathsf{memo})$, where the 32-byte rseed — drawn uniformly at random, fresh for each output note — is the master randomness from which she derives the note's rcm, ψ, and the ephemeral secret esk below, and memo is a fixed 512-byte field of sender-chosen data. Each derivation is one PRFexpand call keyed by rseed and distinguished by a leading domain-separator byte; esk and ψ take $\rho := \mathsf{nf}_{\text{old}}$ (as 32 little-endian bytes) as input — which binds the note's secret randomness to its Action — and rcm, derived last, takes every note field:

$$\begin{aligned}
\mathsf{esk} &:= \mathsf{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x04} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}},\\
\psi &:= \mathsf{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x09} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},\\
\mathsf{rcm} &:= \mathsf{ToScalar}\big(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathsf{pre_rcm})\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}},\\
\mathsf{pre_rcm} &:= \texttt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi),
\end{aligned}$$

where esk is redrawn under a fresh rseed in the rare event it vanishes. This is the Ironwood derivation, the current note format (ZIP-2005; §5.2 gives the exact byte layout and the order's rationale — rcm comes last because its input includes ψ); legacy lead-byte-0x02 notes — all Orchard-pool notes — instead derive $\mathsf{rcm} := \mathsf{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x05} \,\|\, \rho))$ (Remark 5.3). She then runs a one-shot Diffie–Hellman to a *fresh* key:

$$\mathsf{esk}\ \text{(above)}, \quad \mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}, \quad \mathsf{s} := [\mathsf{esk}]\,\mathsf{pk_d}, \quad K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s}, \mathsf{epk}),$$

and seals the plaintext under an AEAD (ChaCha20-Poly1305), $C_{\mathrm{enc}} := \mathsf{Enc}_{K_{\mathrm{enc}}}(\mathsf{np})$. The KDF is BLAKE2b-256 with personalisation `Zcash_OrchardKDF` over $\mathsf{s}^\star \,\|\, \mathsf{epk}^\star$. Because $\mathsf{pk_d} = [\mathsf{ivk}]\mathsf{g_d}$, Bob can reach the same secret from his side as $[\mathsf{ivk}]\,\mathsf{epk} = [\mathsf{ivk}][\mathsf{esk}]\mathsf{g_d} = [\mathsf{esk}]\mathsf{pk_d}$, so, beside the sender (who knows esk), exactly the holder of ivk can recover K_{enc} from the published $(\mathsf{epk}, C_{\mathrm{enc}})$. A second ciphertext C_{out}, sealed under the *outgoing cipher key* $\mathsf{ock} :=$ BLAKE2b-256(`Zcash_Orchardock`, $\mathsf{ovk} \,\|\, \mathsf{cv}^{\mathsf{net}} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk}$) (points in $\star$-encoding; $\mathsf{cv}^{\mathsf{net}}$ is the Action's net value commitment, §2.9), carries $(\mathsf{pk_d}, \mathsf{esk})$ so that the *sender* — or an auditor holding ovk — can re-derive s and read the note as well; because ock binds the Action's own $\mathsf{cv}^{\mathsf{net}}$, cmx, and epk, a C_{out} cannot be transplanted onto another Action. The Action publishes $(\mathsf{epk}, C_{\mathrm{enc}}, C_{\mathrm{out}})$ alongside $\mathsf{cv}^{\mathsf{net}}$, nf, rk, cmx, with rk the randomised validating key (§2.10).

Trial decryption (recipient side). Nothing on chain marks which Action pays Bob, so his wallet *trial-decrypts every* Orchard output: for each it recomputes $\mathsf{s} := [\mathsf{ivk}]\,\mathsf{epk}$, $K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s}, \mathsf{epk})$, and $\mathsf{np} := \mathsf{Dec}_{K_{\mathrm{enc}}}(C_{\mathrm{enc}})$. The AEAD tag makes an incorrect guess fail cleanly (it returns $\perp$), so a successful open *is* the detection that the note was his. He then re-derives the full note from (d, v, rseed) — taking $\rho := \mathsf{nf}_{\mathrm{old}}$ from the same Action — and performs the two checks that close the obvious forgeries: (i) that the sender formed the ephemeral key honestly, $[\mathsf{esk}]\mathsf{g_d} = \mathsf{epk}$ for the esk re-derived from rseed (the ZIP-212 check, defeating a mauled epk); and (ii) that the note is indeed the one the Action committed on chain, $\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)\big) = \mathsf{cmx}$. Only when both hold does he accept. A sender can therefore never make Bob accept a note inconsistent with the commitment.

Wallet synchronisation, including from cold storage. This is the operation a wallet performs when it restores from a seed. From the account's "birthday" height it walks the chain forward and trial-decrypts every Action; a light wallet does so against *compact* blocks, which carry only the first 52 bytes of C_{enc} — those covering the compact note plaintext: the lead byte (1 byte; 0x03 for Ironwood outputs, the only allowed value in that pool — 0x02 appears only when scanning legacy Orchard-pool outputs), d (11 bytes), v (8 bytes), and rseed (32 bytes) — together with epk, cmx, and the Action's nullifier nf, which the light wallet needs twice: it supplies $\rho := \mathsf{nf}_{\mathrm{old}}$ for the cmx recomputation that detects the note, and it is the source of the chain's nullifier set against which spentness is checked below. The wallet records each note that opens *with its leaf position* — the index at which it appended its cmx, read directly off the block — and that position is precisely what the commitment tree of §2.6 consumes: the wallet *marks* the position in its shard tree and can then build the note's authentication path on demand, by the root-only assembly described there, without replaying every commitment. To determine which recovered notes remain spendable it additionally requires nk: it computes each note's nullifier (§2.7) and discards those whose nullifier already appears in the chain's nullifier set. The end state is a set of unspent notes, each with a value, a position, an authentication path, and a nullifier — exactly what the Action prover of §2.11 consumes.

2.9 Conservation of value: value commitments and the binding signature

The fourth requirement: to guarantee that a transaction creates no value, while concealing every amount. The instrument is the additive homomorphism of Pedersen commitments (the *Crypto Guide*): commit to each amount, and verify that the commitments *sum* to the publicly declared net flow — the individual values remain hidden, while their balance is publicly verifiable.

Definition 2.8 (Net value commitment)**.** Each Action commits to its own value change with a Pedersen commitment

$$\mathsf{cv}^{\mathsf{net}} := [\,v_{\mathrm{old}} - v_{\mathrm{new}}\,]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}]\,R^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

where v_{old} is the spent value, v_{new} the created value, $\mathsf{rcv} \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ the commitment randomness (a Pallas scalar), and $V^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"v"})$, $R^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"r"}) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ are nothing-up-my-sleeve points (§2.2), hence independent Pallas generators.

The spender draws each rcv uniformly at random and afresh for every Action, at the moment of bundle assembly. In contrast to the note's rcm, ψ, and esk — all derived deterministically from the note seed rseed (§2.8) — rcv belongs to no note and is never transmitted; its uniformity and secrecy are precisely what make $\mathsf{cv}^{\mathsf{net}}$ hiding, and what keep the aggregate bsk introduced below known only to the transaction author.

We now apply the homomorphism. Summing over the m Actions in a transaction,

$$\sum_{i=1}^{m} \mathsf{cv}^{\mathsf{net},i} = \Big[\textstyle\sum_i (v_{\mathrm{old},i} - v_{\mathrm{new},i})\Big] V^{\mathsf{Orchard}} + [\mathsf{bsk}]\,R^{\mathsf{Orchard}}, \qquad \mathsf{bsk} := \textstyle\sum_i \mathsf{rcv}_i \;\in\; \mathbb{F}_{p_{\mathsf{Vesta}}}.$$

If the transaction is balanced, the bracketed value-component equals the publicly declared net flow $v^{\mathsf{Orchard}}_{\mathrm{balance}}$ (a signed field of the transaction; by convention a positive value is net value flowing *out* of the pool — paying fees or transparent outputs, say). The network subtracts the declared part,

$$\mathsf{bvk} := \sum_i \mathsf{cv}^{\mathsf{net},i} - [\,v^{\mathsf{Orchard}}_{\mathrm{balance}}\,]\,V^{\mathsf{Orchard}} \;\in\; \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

and *if and only if* the transaction balances, the V^{Orchard}-component cancels and $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$ is a public key whose secret bsk the transaction author knows. Proving knowledge of that secret — via a *binding signature* under key bvk — is precisely proving that the accounts balance, with no amount revealed. The binding signature is RedPallas (§2.10) instantiated on the base R^{Orchard}, the value-commitment randomness base, distinct from the spend-authorisation base $G^{\mathsf{SpendAuth}}$. A single signature over the whole transaction (concretely, over the ZIP-244 *sighash* below) additionally binds all its Actions together, so that no one can remove or transplant any of them. From NU6.3 the construction runs once per pool: a v6 transaction carries independent $v^{\mathsf{Orchard}}_{\mathrm{balance}}$ and $v^{\mathsf{Ironwood}}_{\mathrm{balance}}$ fields, and each pool's Actions sum to their own bvk, checked by their own binding signature over the same bases $V^{\mathsf{Orchard}}, R^{\mathsf{Orchard}}$ (ZIP-229; §5.1).

The transaction digest (ZIP-244). The message both Orchard signatures sign — the *sighash* — is not the raw transaction but a non-malleable digest of it, which ZIP-244 assembles as a tree of personalised BLAKE2b-256 hashes, one node per transaction component. Its root is

$$\mathsf{sighash} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \mathsf{branchid},\; d_{\mathrm{hdr}} \,\|\, d_{\mathrm{trans}} \,\|\, d_{\mathrm{Sap}} \,\|\, d_{\mathrm{Orch}}\big),$$

where branchid is the 4-byte identifier of the active network upgrade — so a signature is valid only on the fork it was made for — and each $d_{\bullet}$ is a personalised digest of one pool's data, an absent pool hashing the empty string. A purely shielded Orchard payment leaves the transparent and Sapling branches empty, and then the sighash coincides with the transaction identifier itself.

The Orchard branch commits to the entire bundle,

$$d_{\mathrm{Orch}} := \text{BLAKE2b-256}\big(\texttt{ZTxIdOrchardHash}, d_{\mathrm{C}} \,\|\, d_{\mathrm{M}} \,\|\, d_{\mathrm{N}} \,\|\, \mathsf{flagsOrchard} \,\|\, \mathrm{LE}_{64}(v^{\mathsf{Orchard}}_{\mathrm{balance}}) \,\|\, \mathsf{anchor}\big),$$

through three digests that partition every Action's fields: the *compact* d_{C} over nf, cmx, epk and the first 52 bytes $C_{\mathrm{enc}}[0:52]$; the *memo* d_{M} over the 512-byte $C_{\mathrm{enc}}[52:564]$; and the *non-compact* d_{N} over $\mathsf{cv}^{\mathsf{net}}$,

rk, the tail $C_{\text{enc}}[564:]$ and C_{out}. Folding in every Action's nf, cmx, cv^{net}, rk and ciphertexts beside the shared flagsOrchard, balance and anchor, the sighash pins the exact bundle: no Action can be transplanted into another transaction nor any field altered without breaking it. The SpendAuthSig under each rk and the binding signature under bvk both sign this single value. The v6 form of the digest appends the Ironwood component digest as a fifth child, d_{Irw}, under revised personalisations, and moves the Sapling, Orchard, and Ironwood anchors from effecting data to authorizing data (§5.1; ZIP-229; `librustzcash` `zcash_primitives`, `transaction/sighash_v6.rs`).

2.10 Authorisation of the spend: RedPallas

One capability remains: the spender must prove that they are *authorised* to spend the note — that they hold ask — and must do so without linking this spend to any other spend by the same key. RedPallas, a re-randomisable Schnorr signature, provides exactly this.

It is the Schnorr template of the *Crypto Guide* on Pallas, with hash H^* (BLAKE2b-512 reduced mod p_{Vesta}). The novel ingredient is *key re-randomisation*: the spender draws a randomiser $\alpha \in \mathbb{F}_{p_{\mathsf{Vesta}}}$ uniformly at random, fresh for every Action (RedPallas's GenRandom: hash 80 uniform bytes with H^*, which makes α statistically uniform on $\mathbb{F}_{p_{\mathsf{Vesta}}}$), and sets

$$\mathsf{rsk} := \mathsf{ask} + \alpha \in \mathbb{F}_{p_{\mathsf{Vesta}}}, \qquad \mathsf{rk} := \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\, G^{\mathsf{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}).$$

The spender publishes the *randomised* validating key rk, not the real ak, and signs with rsk. Because α is fresh per spend, rk is uniformly random and unlinkable across spends, yet remains a valid key for the underlying authority. The circuit enforces that rk is a re-randomisation of the witnessed ak (check A6) and that this ak is the key bound into the note's address (check A7), so that a valid Action proves the spender knows ask — without ever revealing ak.

2.11 A single shielded payment, end to end

The components now assemble into a transaction. Consider Alice paying Bob 5 ZEC ($v = 5 \cdot 10^8$ zatoshi) from a note worth 5 ZEC. The unit of shielded transfer is an *Action*, which simultaneously spends one old note and creates one new note (a deliberately fixed shape that conceals whether a given Action is "really" a spend, an output, or both). This payment is one Action.

1. Bob hands Alice an address. Bob derives a diversified address $(d, \mathsf{pk_d})$ from his ivk (§2.4) and gives it to Alice. It reveals nothing and is unlinkable to his other addresses.

2. Alice builds the Action. Alice owns an old note $\mathbf{n}_{\text{old}}$ worth 5 ZEC, whose commitment $\mathsf{cmx}_{\text{old}}$ is a leaf of the tree. She:

- computes its nullifier nf_{old} (§2.7) — she will publish this to retire the note;
- constructs Bob's new note $\mathbf{n}_{\text{new}} := (\mathsf{addr}_{\text{Bob}}, 5 \cdot 10^8, \rho_{\text{new}}, \psi_{\text{new}}, \mathsf{rcm}_{\text{new}})$ with $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$, and its commitment $\mathsf{cmx}_{\text{new}}$ (§2.5);
- forms the value commitment cv^{net} with $v_{\text{old}} - v_{\text{new}} = 5 \cdot 10^8 - 5 \cdot 10^8 = 0$ (§2.9);
- selects a randomiser α and forms rk (§2.10);
- reads off a recent anchor and her note's Merkle path (§2.6).

3. Alice proves the Action statement. She runs the Halo 2 prover on the relation of §2.14, with public inputs $(\mathsf{anchor}, \text{cv}^{\text{net}}, \mathsf{nf}_{\text{old}}, \mathsf{rk}, \mathsf{cmx}_{\text{new}})$ and her note, keys, path, and randomness as private witness. The proof attests, in one shot and revealing nothing: the old note is in the tree (A3), its nullifier follows correctly (A5), she is authorised to spend it (A6, A7), the spent and created notes are well-formed (A1, A2), and the value commitment is consistent (A4).

4. Alice assembles and signs the transaction. She encrypts $\mathbf{n}_{\text{new}}$ to Bob (so that only Bob, via his ivk, can detect and open it; §2.8), attaches a SpendAuthSig under rk, and signs the whole bundle with the binding signature under bvk (§2.9), which proves balance; both signatures sign the ZIP-244 transaction sighash, welding the Action into this transaction.

5. The network verifies. Consensus checks: it recognises the anchor; nf_{old} is not already in the nullifier set (no double-spend); the Halo 2 proof verifies; the SpendAuthSig verifies under rk; the binding signature verifies under the recomputed bvk (balance). If all pass, it adds nf_{old} to the nullifier set and appends $\mathsf{cmx}_{\text{new}}$ as a new tree leaf.

6. Bob receives. Scanning with his ivk, Bob trial-decrypts the ciphertext (§2.8), recovers $\mathbf{n}_{\text{new}}$, and now owns a 5-ZEC note he can later spend by repeating the entire procedure. The chain has gained one nullifier and one commitment; an observer learns that *an* Orchard payment occurred and nothing else — not sender, recipient, or amount.

That sequence constitutes the entire protocol. The formal Action statement below is precisely the list of the in-circuit checks step 3 invokes.

2.12 How value enters the pool

The walkthrough moved value already inside the pool. Value *enters* through three doors, each public in amount: the two below, and migration from another shielded pool (§5.4).

Shielding transactions. The instrument is the sign of the balancing value: by the convention of §2.9, a positive $v^{\mathsf{Orchard}}_{\text{balance}}$ is net value leaving the pool, so a *shielding* transaction is one whose transparent inputs fund a negative $v^{\mathsf{Orchard}}_{\text{balance}}$: its Actions spend dummy notes ($v_{\text{old}} = 0$, the anchor check A3 gated off, §2.15) and create real ones, and the binding signature of §2.9 proves the books balance. An observer learns the entering amount — the balancing field is a cleartext transaction field — but not the receiving note or address. The deployed construction path is function `propose_shielding` (`zcash_client_backend`, `data_api/wallet.rs` and `input_selection.rs`): it sweeps the given source addresses' UTXOs, at zero confirmations under the default policy, and wallets auto-shield transparent receipts to internal keys.

Shielded coinbase (ZIP-213). Since Heartwood, new issuance may enter directly: "[c]oinbase transactions MAY contain Sapling outputs", extended at NU5 to the Orchard pool and re-scoped at NU6.3 — from then coinbase may carry "Sapling-pool and/or Ironwood-pool outputs", and "it is only possible to mine to an Orchard address by sending the funds to the *Ironwood pool*" (ZIP-213). Two riders keep this auditable and consistent. Coinbase maturity is amended to bind transparent outputs only (the rule the *Consensus Guide* reads through its theorem, §"Reorg rails, maturity, and the finality floor"), and every shielded coinbase output "MUST have valid note commitments" when recovered under the *all-zero* outgoing viewing key: coinbase shielding is publicly decryptable by construction, so anyone can verify what issuance entered which pool, while the notes spend exactly like any others afterwards.

The ledger of crossings. Every entry and exit is metered by ZIP-209's pool balances: for each shielded pool, the *chain value pool balance* is the negation of the sum of that pool's balancing fields over the whole chain (for Sprout, the sum of `vpub_old` − `vpub_new`), and a block is invalid if it would drive any pool balance — shielded, transparent, or the deferred fund (ZIP-2001's lockbox, a chain value pool accruing a portion of each block subsidy for later disbursement) — negative, or total supply past MAX_MONEY. Entry, transfer, and exit therefore reconcile publicly at pool granularity while individual notes stay hidden; the Ironwood instantiation, and the one-way turnstile that closes the Orchard pool's entry door entirely, are §5.4's.

2.13 What a node verifies, without ever observing the note

Note that none of the previous subsection involved the *network*. A validating node holds no viewing key, cannot trial-decrypt, and so never learns the sender, recipient, or amount of any Action. What it *can* do is confirm that the hidden data is internally consistent — and for that it relies entirely on the proof, not on the note.

What the node receives is an Orchard-protocol *bundle*, the unit into which the transaction format groups one pool's data: the per-Action public fields; one anchor shared by every Action in the bundle; a *single* aggregate Halo 2 proof covering all of the bundle's Actions; and one SpendAuthSig per Action. The declared net flow $v_{\text{balance}}^{\text{Orchard}}$ and the binding signature appear once per bundle, and the flags enableSpends, enableOutputs, and — from NU6.3 — enableCrossAddress (bit 2 of the flags byte, reserved as 0 in v5 transactions before NU6.3) are likewise bundle-level fields. The current carrier is the v6 format (ZIP-229): the v5 format's Orchard component (ZIP-225) plus a separate Ironwood component holding that pool's bundle (flagsIronwood, valueBalanceIronwood, anchorIronwood, proofsIronwood, vSpendAuthSigsIronwood, bindingSigIronwood; §5.1). Transaction versions v4, v5, and v6 all remain valid from NU6.3 onward, and the Orchard-pool sealing rules of §5.1 bind regardless of version (ZIP-229). Using only the public fields (anchor, cv^{net}, nf_{old}, rk, cmx) of each Action, a node checks:

- the bundle's *Halo 2 proof* of the Action statement (§2.14) — this forces each spent note of nonzero witnessed value to be a genuine tree leaf (a zero-value dummy spend skips the membership check by A3's gate, by design), the nullifiers and new commitments to follow correctly, and the keys to align;
- the *anchor* is a Merkle root the node has seen on its accepted main chain (§2.6);
- nf_{old} is *not already* in the nullifier set — the one check that cannot sit inside a single proof, because it concerns the global history (this prevents a double-spend);
- the *spend-authorisation signature* verifies under rk and the *binding signature* under the recomputed bvk (§§2.9–2.10), settling authority and balance.

If every Action passes, the node performs the two state changes of step 5 — recording each nf_{old}, appending each $\mathsf{cmx}_{\text{new}}$ — which permit the next spender to prove membership and make a second spend of the same note fail. Thus the SNARK together with the two signatures enforces a payment's validity, never the inspection of the note: the node confirms that the accounts balance and that no one forged anything *while learning nothing about the note itself*. This is the precise sense in which the knowledge soundness of the Action SNARK (§3) performs all the work — the network trusts the proof exactly because a real witness extractably backs a valid one.

2.14 The Action statement, formally

Definition 2.9 (Orchard Action statement, Ironwood circuit (NU6.3))**.** The Action SNARK proves, with public inputs

$$(\text{anchor},\ \mathsf{cv}^{\text{net}},\ \mathsf{nf}_{\text{old}},\ \mathsf{rk},\ \mathsf{cmx},\ \text{enableSpends},\ \text{enableOutputs},\ \text{disableCrossAddress})$$

and the note, keys, Merkle path, randomiser α, and the value-commitment trapdoor rcv as private witness, the conjunction:

[A1] Old-note commitment integrity — the spent note is well-formed:

$$\mathsf{cm}_{\text{old}} = \mathsf{NoteCommit}_{\mathsf{rcm}_{\text{old}}}\big(\mathsf{g}_{\mathsf{d}_{\text{old}}}^{\star} \| \mathsf{pk}_{\mathsf{d}_{\text{old}}}^{\star} \| \mathrm{LE}_{64}(v_{\text{old}}) \| \mathrm{LE}_{255}(\rho_{\text{old}}) \| \mathrm{LE}_{255}(\psi_{\text{old}})\big)$$

— or NoteCommit returns ⊥, the specification's escape hatch for Sinsemilla's incomplete-addition exceptions (the remark following this definition).
[A2] New-note commitment integrity, with $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$: $\mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}_{\text{new}}}(\ldots)\big) \in \{\mathsf{cmx}, \bot\}$ (with $\mathsf{Extract}(\bot) := \bot$) — the created note is well-formed and its uniqueness seed chains to the spent note's nullifier (Faerie resistance).
[A3] Membership, gated by v_{old} — the spent note is a real leaf: recomputing the Merkle root from $\mathsf{Extract}(\mathsf{cm}_{\text{old}})$ up the witness path of length 32 gives root, and $v_{\text{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$. (The gate by v_{old} permits a zero-value "dummy" spend to skip the membership check, which is how an Action pads with no real input.)
[A4] Net-value commitment integrity — the amounts are consistent and in range: with witnessed (magnitude, sign), $v_{\text{old}} - v_{\text{new}} = \text{magnitude} \cdot \text{sign}$, $\mathsf{cv}^{\mathsf{net}} = [v_{\text{old}} - v_{\text{new}}]V^{\mathsf{Orchard}} + [\mathsf{rcv}]R^{\mathsf{Orchard}}$, $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$, $\text{sign} \in \{-1, +1\}$.
[A5] Nullifier integrity — the published nullifier is indeed this note's:

$$\mathsf{nf}_{\text{old}} = \mathsf{Extract}\big([F_{\mathsf{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}]\, G^{\mathsf{nf}} + \mathsf{cm}_{\text{old}}\big).$$

[A6] Spend authority — the published key is a re-randomisation of the note owner's: $\mathsf{rk} = \mathsf{ak} + [\alpha]G^{\mathsf{SpendAuth}}$.
[A7] Address integrity — the keys, address, and nullifier key all belong to one identity: $\mathsf{ivk} = \bot$, or $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ and $\mathsf{pk}_{\mathsf{d_{old}}} = [\mathsf{ivk}]\mathsf{g}_{\mathsf{d_{old}}}$.
[A8] / [A9] Spend / output gating: $v_{\text{old}} \cdot (1 - \mathsf{enableSpends}) = 0$ and $v_{\text{new}} \cdot (1 - \mathsf{enableOutputs}) = 0$. The transaction builder sets these bundle-level flags; consensus rules constrain them — coinbase transactions, for instance, must set $\mathsf{enableSpends} = 0$ (ZIP-229). Spending Ironwood notes is otherwise permitted; the $\mathsf{enableSpends} = 0$ requirement is coinbase-scoped, not a pool-wide prohibition (protocol specification § 7.1, transaction consensus rules).
[A10] Cross-address gating — when $\mathsf{disableCrossAddress} \neq 0$, the created note pays the spent note's own receiver: $\mathsf{g_{d_{new}}} = \mathsf{g_{d_{old}}}$ and $\mathsf{pk_{d_{new}}} = \mathsf{pk_{d_{old}}}$ as full curve points, enforced by four product constraints of the A3 shape, one per affine coordinate (Definition 5.4, §5.3, gives the rows, the instance index — the flag is the tenth public input, at index 9 — and the wire encoding). Consensus requires every post-NU6.3 Orchard-pool Action to set the flag (§5.1).

Remark 2.10 (Three circuit versions, one owning ZIP pending)**.** The relation above is the third Action-circuit version: the deployed implementation distinguishes the insecure NU5 original, the NU6.2 correction (ZIP-257, Final), and the NU6.3 Ironwood circuit, and only the last constrains the flag (`OrchardCircuitVersion::supports_cross_address_restriction`, released crate `orchard` 0.15.0 `src/circuit.rs`). The legacy NU5/NU6 statement is the relation A1–A9 alone, over the nine-element public-input tuple ending at enableOutputs. Consensus selects the verifying key by branch (ZIP-258), so from NU6.3 every Action — in either pool, whatever the transaction version — verifies against the Ironwood key. The circuit change's owning ZIP does not yet exist as text: ZIP-2006 is a Reserved stub, and the flag's encoding, its consensus rule, and the verifying-key selection are *specified* in ZIP-229/258 (§5.3 states what remains reconstructable only from the implementation).

A1, A2, and A7 take the specification's ⊥-weakened forms because the circuit cannot exclude Sinsemilla's incomplete-addition exceptional cases; the relation proved is the disjunction, not the plain conjunction. The weakening costs only a negligible soundness term: producing a ⊥ case is itself a non-trivial DL relation among the GroupHash generators (the closing sentence of the proof of Proposition 3.1), so no efficient prover exhibits one.

Each of A1–A7 is one entry from the four-requirement map of §2.1: A1–A2 represent notes, A3 proves membership, A5 (with the chain's nullifier set) prevents double-spending, A4 conserves value, and A6–A7 authorise the spender; A8–A10 stand outside the map as consensus-level gating constraints. This is the relation $\mathcal{R}_{\mathsf{Orchard}}$. Every Action is an instance of it; Halo 2 produces one aggregate proof *per bundle*, covering all of the bundle's Actions, while each Action carries its own SpendAuthSig. §2.15 sketches how the Action circuit arithmetises this relation, and §3 shows how soundness of

the relation yields the protocol's security properties.

2.15 The Action circuit: arithmetisation of the relation

The Action statement of §2.14 is a *relation*; what Alice actually executes is a *circuit* that proves membership in it in zero knowledge. Halo 2 (the *Halo 2 Guide*) proves satisfiability of a fixed PLONKish circuit, so the task of the application layer is to encode the conjunction A1–A10 as one such circuit — the *Action circuit*, identical for every Action — supplied the note, keys, Merkle path, randomiser α, and the value-commitment trapdoor rcv as private witness. We sketch this encoding here; the gate-level details appear in the *halo2 Book* and the *Orchard Book*.

We assemble the circuit from a small set of reusable *gadgets* ("chips"), each a pre-wired block of custom gates and lookups realising one operation. The Orchard designers *chose* the primitives so that these gadgets are inexpensive under PLONKish arithmetisation; this motivates the use of Sinsemilla and Poseidon rather than a bit-oriented hash. How any such gadget is *built* — how its gates, lookups, and cell assignments are declared, and how synthesis turns the filled columns into the polynomials the proof commits to — is the subject of the *Halo 2 Guide*'s section "From statement to circuit: arithmetisation in practice"; here we describe what each gadget enforces.

The ECC gadget. Implements the Pallas group law with the *complete* addition formulas (no edge-case failures; the *Math Guide*), together with variable- and fixed-base scalar multiplication (the former a double-and-add ladder of incomplete-addition rounds finished by a complete tail, the latter using precomputed fixed-base tables; the j-invariant-0 endomorphism serves only the recursion layer, where it cheapens multiplication by verifier challenges, not this circuit) and on-curve witnessing. It realises every group operation in the statement: $\mathsf{ak} = [\mathsf{ask}]\,G^{\mathsf{SpendAuth}}$, the re-randomisation $\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}$ (A6), the transmission key $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ (A7), the net value commitment $\mathsf{cv}^{\mathsf{net}} = [\,v_{\mathrm{old}} - v_{\mathrm{new}}\,]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}]\,R^{\mathsf{Orchard}}$ (A4), and the point $[\,F_{\mathsf{nk}}(\rho) + \psi\,]\,G^{\mathsf{nf}} + \mathsf{cm}$ inside the nullifier (A5).

The Sinsemilla gadget. Implements Sinsemilla (§2.3) by realising its generator table $P[m]$ as a Halo 2 *lookup* (the lookup argument of the *Halo 2 Guide*): each 10-bit message chunk costs one table lookup and two incomplete additions. On it rest the two composed commitment gadgets the Orchard Book names explicitly — NoteCommit, the in-circuit NoteCommit proving A1 and A2, and CommitIvk, the in-circuit short commitment proving $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ in A7 — each bundling the Sinsemilla rounds with the bit-decomposition and canonicity checks of their field inputs.

The Merkle-path gadget. Stacks 32 layer-tagged MerkleCRH hashes (themselves Sinsemilla short hashes, §2.6); at each layer a conditional swap orders the running node and its witnessed sibling by one bit of the leaf position, and the gadget asserts the recomputed root equal to the public anchor — check A3, gated by v_{old} so that a dummy spend may skip it.

The Poseidon gadget. Implements the Poseidon permutation (*Crypto Guide*), whose only non-linearity is $x \mapsto x^5$, to evaluate the nullifier PRF $F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho)$ cheaply in-circuit; its output feeds the ECC gadget for A5.

Decomposition, range, and boolean gadgets. Lookup-based gadgets split field elements into the bit-chunks the other gadgets consume and range-check values: that $v_{\mathrm{old}}, v_{\mathrm{new}} \in [0, 2^{64})$ and $\mathrm{sign} \in \{-1, +1\}$ (A4), and that point and field encodings are canonical. (The flags $\mathsf{enableSpends}$, $\mathsf{enableOutputs}$, and $\mathsf{enableCrossAddress}$ (bit 2) are verifier-supplied public inputs — ZIP-229's flags bytes, $\mathsf{flagsOrchard}$ and $\mathsf{flagsIronwood}$, fix them to bits outside the circuit — and enter the circuit only through the plain-gate products of A8–A10, the third negated as the Ironwood circuit's tenth public input $\mathsf{disableCrossAddress}$, §5.3.)

Structure of the proof. These gadgets compose into a single fixed circuit whose native field is the Pallas base field $\mathbb{F}_{p_{\mathsf{Pallas}}}$ — the field in which Pallas coordinates reside, so that the ECC gadget runs natively — proved with the Halo 2 inner-product argument over the Pasta cycle. The circuit uses $2^{11} = 2048$ rows (the *Halo 2 Guide*'s circuit-size exponent $k = 11$ — not to be confused with Sinsemilla's chunk width $k = 10$, Construction 2.1), with ten advice columns beside the fixed columns and the Sinsemilla lookup table (the *Halo 2 Guide*). Every Action, whatever it spends or creates, is one instance of this same circuit; a bundle's Actions are proved together in one aggregate proof that a node checks (§2.13) from the public inputs alone, and batch verification — the deferred linear MSMs merged across proofs (the *Halo 2 Guide*) — lets a node verify many such proofs at amortised cost approaching that of a single verification.

3 End-to-end security: derivation of Orchard's properties from the SNARK

The Halo 2 SNARK on the Action relation, together with the algebraic structure of the Orchard primitives, produces the security properties that render shielded payments functional. We establish those connections below.

3.1 Guarantees provided by a valid Action proof

A SNARK π accepted on public inputs $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\mathsf{old}}, \mathsf{rk}, \mathsf{cmx}, \ldots)$ testifies, with overwhelming probability, that the prover knows a witness satisfying A1–A10. By knowledge soundness of Halo 2 (the *Halo 2 Guide*), an extractor can recover this witness from any successful prover. The soundness-side properties below (nullifier uniqueness, spend authority, balance) each follow from one or more conjuncts of A1–A10 plus the algebraic properties of the primitives; §3.2 first supplies the primitive reduction those arguments share, and privacy rests instead on the zero knowledge of the SNARK (§3.6) together with the DDH, Poseidon-PRF and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.2 Reduction of Sinsemilla collision-resistance to DL on Pallas

> **Proposition 3.1** (Sinsemilla CR $\Leftarrow$ DL). *A collision $M \neq M'$ between messages of the same chunk count ℓ with $\mathsf{Sinsemilla}_D(M) = \mathsf{Sinsemilla}_D(M')$ yields a non-trivial DL relation among the generators $\{P[0], \ldots, P[2^k - 1]\}$.*

Sketch. Unfolding the Sinsemilla update (in the case no incomplete-addition exception occurs), with ℓ the number of k-bit chunks:

$$\mathsf{Sinsemilla}_D(M) = 2^\ell \cdot Q + \sum_{i=1}^{\ell} 2^{\ell - i} \cdot P[m_i].$$

A collision gives $\sum_i 2^{\ell-i}(P[m_i] - P[m_i']) = 0$, i.e. a non-trivial linear relation over $\mathbb{F}_{p_{\mathsf{Vesta}}}$ on the generators. Since hash-to-curve modelled as a random oracle samples these independently, finding such a relation is the multi-generator discrete-log relation problem, and no efficient adversary produces one without solving DL (the *Crypto Guide* gives this reduction in its Pedersen-hash collision analysis). For differing chunk counts $\ell \neq \ell'$, the unfolded equality instead gives $(2^\ell - 2^{\ell'})\,Q + \sum_i \cdots = \mathcal{O}$ with $2^\ell \not\equiv 2^{\ell'} \pmod{p_{\mathsf{Vesta}}}$ for all admissible lengths, i.e. a non-trivial DL relation among $\{Q\} \cup \{P[0], \ldots, P[2^k - 1]\}$; since Q is itself a GroupHash output modelled as an independent random generator, this case reduces to DL identically. The incomplete-addition exceptions are themselves non-trivial DL relations among the same generators and so also negligible. □

Sinsemilla CR is the primitive ingredient for note-commitment binding: two distinct notes cannot share a commitment, so A1 and A2 are genuine integrity statements about $\mathsf{cm}_{\mathrm{old}}$ and $\mathsf{cm}_{\mathrm{new}}$. Combined with knowledge soundness, this entails that the prover knows the specific opening of each commitment.

3.3 Nullifier uniqueness

Proposition 3.2 (Nullifier uniqueness)**.** *Two distinct notes $\mathbf{n} \neq \mathbf{n}'$ cannot share a nullifier except with probability negligible in the DL hardness of Pallas (treating Poseidon outputs as random scalars under the PRF assumption).*

Sketch. A collision $\mathsf{nf}(\mathbf{n}) = \mathsf{nf}(\mathbf{n}')$ in x-coordinate gives $[F_{\mathsf{nk}}(\rho) + \psi]G^{\mathsf{nf}} + \mathsf{cm} = \pm([F_{\mathsf{nk}}(\rho') + \psi']G^{\mathsf{nf}} + \mathsf{cm}')$, a non-trivial linear relation among $G^{\mathsf{nf}}, \mathsf{cm}, \mathsf{cm}'$. Expanding $\mathsf{cm}, \mathsf{cm}'$ via their known openings (extracted by knowledge soundness, or supplied by the adversary who forged the notes) over the Sinsemilla generators and the blinding base H_D turns this into a known non-trivial DL relation among the independent GroupHash outputs $\{G^{\mathsf{nf}}, Q, P[0], \ldots, P[2^k - 1], H_D\}$ — distinctness of the notes forces some coefficient mismatch — and producing such a relation reduces to DL on Pallas as in Proposition 3.1. □

A5 (nullifier integrity) plus knowledge soundness ensures that $\mathsf{nf}_{\mathrm{old}}$ in the public inputs is genuinely the nullifier of some specific committed note, not an arbitrary value. The chain's duplicate-nullifier rejection then prevents double-spending.

3.4 RedPallas unforgeability and re-randomisation

Proposition 3.3 (RedPallas EUF-CMA $\Leftarrow$ DL)**.** *RedPallas is EUF-CMA-secure assuming DL on Pallas in the random-oracle model. The re-randomisation extension preserves security: a forger that, given* ak *and signatures under re-randomised keys, produces a valid signature under* $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ *for an α of its own (possibly* ak*-dependent) choice yields a DL solver for* ak*.*

Sketch. The standard forking argument (the *Crypto Guide*, via the Bellare–Neven general forking lemma) reduces a Schnorr forger to a DL adversary, and it covers re-randomisation because RedPallas is key-prefixed: the challenge $H^*(R \,\|\, \mathsf{rk} \,\|\, m)$ hashes the verification key, so response-shifting cannot transport signatures between keys. The reduction embeds the DL challenge as $\mathsf{ak} := X$. It answers signing queries under any $\mathsf{rk}_i = \mathsf{ak} + [\alpha_i]\, G^{\mathsf{SpendAuth}}$ without knowing any secret, via the honest-verifier zero-knowledge simulator with random-oracle programming: sample c, s uniformly, set $R := [s]\, G^{\mathsf{SpendAuth}} - [c]\, \mathsf{rk}_i$, and program $H^*(R \,\|\, \mathsf{rk}_i \,\|\, m) := c$, aborting on programming collisions exactly as in the *Crypto Guide*'s Schnorr EUF-CMA theorem. It then forks the adversary at the forgery's critical query $H^*(R^* \,\|\, \mathsf{rk}^* \,\|\, m^*)$, obtaining two accepting transcripts that share R^* — and, since both forks share the critical query input, the same rk^*, hence the same α^* — with distinct challenges; 2-special soundness extracts rsk^*, the discrete logarithm of rk^*, and the output $\mathsf{ask} = \mathsf{rsk}^* - \alpha^*$, with α^* the adversary-supplied randomiser, solves DL for ak. Separately, and only for honest signer-chosen uniform α: sampling α at random and back-deriving $\mathsf{ak} := \mathsf{rk} - [\alpha]\, G^{\mathsf{SpendAuth}}$ reproduces the view of $(\mathsf{ak}, \alpha, \mathsf{rk})$ exactly, so honest re-randomisations are distributed as plain Schnorr keys — the unlinkability statement of the *Crypto Guide*, which does not by itself cover adversary-chosen α. □

A6 (spend authority) plus knowledge soundness ensures the prover knows α relating rk to ak; combined with the SpendAuthSig under rk at the transaction layer, this enforces that the spender knows ask corresponding to ak, hence to the address $\mathsf{pk}_{\mathsf{d}_{\mathrm{old}}}$ via A7.

3.5 Balance preservation via the Pedersen homomorphism

The combination of A4 (per-Action value commitment integrity), the Pedersen homomorphism on $\mathsf{cv}^{\mathsf{net}}$, and the binding signature on bvk gives end-to-end balance:

> **Proposition 3.4** (Balance). *Any accepted Orchard bundle has* $\sum_i (v_{\text{old},i} - v_{\text{new},i}) = v_{\text{balance}}^{\mathsf{Orchard}}$ *except with probability negligible in the DL hardness of the Pasta curves in the random-oracle model (Vesta for the knowledge soundness of the Action SNARK supplying the A4 openings; Pallas for the binding-signature extraction and the independence of the value-commitment bases).*

Sketch. By A4 and knowledge soundness, each $\mathsf{cv}^{\mathsf{net},i}$ commits to the true per-Action delta $v_{\text{old},i} - v_{\text{new},i}$. The bundle sum $\sum_i \mathsf{cv}^{\mathsf{net},i}$ equals $[\sum_i (v_{\text{old},i} - v_{\text{new},i})]V^{\mathsf{Orchard}} + [\sum_i \mathsf{rcv}_i]R^{\mathsf{Orchard}}$ by homomorphism. Consensus computes bvk and checks the binding signature; a valid binding signature is a Fiat–Shamir proof of knowledge of $\log_{R^{\mathsf{Orchard}}} \mathsf{bvk}$, so the forking extraction in the proof of Proposition 3.3 recovers bsk with $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$; combined with the openings extracted via A4, a non-zero V^{Orchard}-component of bvk would yield a discrete-log relation between the independent bases V^{Orchard} and R^{Orchard}, contradicting DL — hence the value-component equals $[v_{\text{balance}}]V^{\mathsf{Orchard}}$ and the total value flow matches the declared transparent balance. □

This is the fundamental conservation property: shielded transactions cannot mint zatoshi out of nothing.

3.6 Zero knowledge of the SNARK

Halo 2 is statistical zero knowledge in the random-oracle model (the *Halo 2 Guide*'s zero-knowledge analysis). The simulator on input the instance $(\mathsf{anchor}, \mathsf{cv}^{\mathsf{net}}, \mathsf{nf}_{\text{old}}, \mathsf{rk}, \mathsf{cmx}, \ldots)$ operates as follows:

1. Sample the Fiat–Shamir challenges $\theta, \beta, \gamma, y, x, x_1, x_2, x_3, x_4$ and the IPA challenges $u_1, \ldots, u_k$ ($k = 11$ rounds, the circuit-size exponent) uniformly, programming the random oracle to return these on the relevant transcript prefixes.
2. Sample the final IPA scalar a uniformly.
3. Use the freedom in the blinding terms ℓ_j, r_j (added at each IPA round) and the polynomial blinders (added to each committed prover polynomial: the advice columns in phase 1, the running products in phase 3, and the quotient chunks in phase 4) to satisfy the verifier's equations algebraically without knowing the witness.

The resulting transcript is statistically indistinguishable from a real proof; the only gap from perfect ZK is the random-oracle programming, which a verifier that observes only the transcript cannot detect.

ZK supplies the proof's share of Orchard's privacy: the SNARK transcript reveals nothing beyond the public inputs of each Action. That the published data leak nothing further is a separate, computational matter: the nullifier — itself a public input (§2.7) — hides its note under the PRF security of Poseidon, and the epk and ciphertexts published alongside the public inputs (§2.8) hide recipient and value under the DDH and KDF/AEAD assumptions that Theorem 3.5(iv) adds.

3.7 Composition of end-to-end soundness

The full SNARK soundness composes three ingredients:

- *PIOP soundness.* Schwartz–Zippel error per polynomial-identity check, summed across constraints. For $\mathbb{F} = \mathbb{F}_{p_{\mathsf{Pallas}}}$ with $|\mathbb{F}| \approx 2^{254}$ and constraint degree $\leq d_{\max}$, the error per check is $\leq d_{\max}\, n/|\mathbb{F}| \approx d_{\max} \cdot 2^{-243}$ (the gate polynomial composes degree-$d_{\max}$ gates with column polynomials of degree

$n-1, n = 2^{11}$; a cheating prover's recombined quotient $h \cdot Z_H$ has degree $< d_{\max} n$), overwhelmingly small.

- *PCS extractability.* Reduces to DL on the IPA curve (the *Halo 2 Guide*), with the accumulation analysis (the *Halo 2 Guide*) covering the deferred-MSM batching by which a node amortises verification across proofs (§2.15); the same analysis would carry soundness across recursion, which Orchard's deployment does not use.
- *Fiat–Shamir soundness in the ROM.* The *Crypto Guide*'s FS theorem covers Σ-protocols, losing roughly a factor Q in the prover's RO-query budget; for the $\log d$-round Halo 2 transcript the naive multi-round analysis loses $q^{O(\log d)}$ (the *Halo 2 Guide*), a bound vacuous at $Q = 2^{80}$. The negligible-error conclusion rests on the tighter treatments the *Halo 2 Guide* presents: BCMS20's direct ROM+DL analysis, or Ghoshal–Tessaro's tight Fiat–Shamir bound in the AGM.

Each step's soundness error is negligible at Orchard's parameters ($|\mathbb{F}| \approx 2^{254}$, $\lambda = 127$ under the group-order-$\approx 2^{2\lambda}$ convention; effectively ≈ 126 bits after the automorphism speedups, the *Crypto Guide*); the union bound across steps is itself negligible. Combined with the application-level reductions (Propositions 3.1–3.4), the full chain gives:

> **Theorem 3.5** (Orchard end-to-end security, informal)**.** *Under DL on both Pasta curves (Pallas for the application-level primitives, Vesta for the IPA commitment) in the random-oracle model — and, for property (iv), additionally DDH on Pallas, the PRF security of Poseidon (§2.7), and the KDF/AEAD securing the note ciphertext (§2.8) — no efficient adversary can (i) produce an accepted Action without knowing a witness satisfying A1–A10 — in particular, any Action whose witnessed v_{old} is nonzero must spend a genuine leaf of the commitment tree (zero-value dummy spends, A3's gate, are accepted by design), (ii) cause two Actions to share a nullifier without spending the same note twice, (iii) cause a bundle's declared transparent balance $v_{\text{balance}}^{\text{Orchard}}$ to differ from its true net value flow $\sum_i (v_{\text{old},i} - v_{\text{new},i})$ — in particular, to exceed it, which would create value (Proposition 3.4), or (iv) link a published Action to any specific recipient or value beyond what is explicit in the public inputs.*

The four properties are, respectively, soundness of the Action SNARK, nullifier uniqueness, balance preservation, and zero knowledge. The first three reduce (informally) to DL on the Pasta curves in the ROM (Pallas for the application-level reductions, Vesta for the IPA extraction). For the fourth (privacy), the zero knowledge of the SNARK transcript is statistical — given the perfectly-hiding Pedersen and Sinsemilla blinders and the random-oracle programming, it does not rest on DL — but full unlinkability of an Action additionally rests on the DDH assumption on Pallas and the PRF security of Poseidon (§2.7) and on the KDF/AEAD securing the note ciphertext (§2.8), and is therefore computational. The asymmetry within the proof itself is typical of the discrete-log SNARK family: soundness is computational (a cheating prover is only *bounded*, not impossible), whereas the zero knowledge of the transcript is information-theoretic.

4 The Zcash proof-system landscape

This section closes with a survey of the proof systems Zcash has used across its three generations of shielded payments. Concrete trade-offs drove each transition: trusted setup risk, recursion compatibility, post-quantum posture, and on-chain efficiency.

4.1 Sprout (2016–present, deprecated)

The original shielded protocol used a *BCTV14* SNARK (Ben-Sasson, Chiesa, Tromer, Virza) over the pairing-friendly curve BN254, with constant-size proofs (296 bytes) and constant verification time. Its parameters came from a per-circuit trusted setup: the 2016 ceremony, a six-party multi-party computation that generated the BCTV14 proving and verification keys for the fixed Sprout circuit, sound

provided at least one participant destroyed their secret share. Sprout demonstrated that zk-SNARK shielded payments were feasible at scale, but it remained a transitional design: the prover was slow, and the circuit hashed with SHA-256 — a bit-oriented hash whose boolean operations are non-native to arithmetic constraints, hence costly in-circuit. Decisively, a flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; the pool was deprecated and shielded Sprout value was migrated to later pools.

4.2 Sapling (2018–present)

Sapling (the second network upgrade) replaced this stack with Groth16 — 192-byte proofs, three-pairing verification — over BLS12-381, with the embedded curve Jubjub serving the in-circuit group operations and a Pedersen hash serving the Merkle tree. The result was a far faster prover, a smaller proof, and cheaper in-circuit hashing. Two structural limitations remained. Groth16 still requires a per-circuit trusted setup — the 2018 Sapling MPC, whose universal first phase was the Powers of Tau ceremony with many participants — and no known elliptic-curve cycle extends BLS12-381, so the proving system admits no efficient recursion.

4.3 Orchard (2022–present, NU5; Ironwood pool from NU6.3)

Orchard (NU5) replaced the Sapling stack with the Halo 2 / Pasta stack developed in the *Halo 2 Guide*. The principal choices are:

- Halo 2: PLONKish PIOP, IPA-PCS, accumulation-friendly. No trusted setup.
- Pasta cycle: Pallas as the application curve, with Vesta as the proving curve in the IPA layer. Designed for recursion.
- Sinsemilla hash: an algebraic hash whose collision resistance reduces to DL on Pallas — an assumption the protocol already requires elsewhere — and whose chunk-table structure maps directly onto Halo 2's lookup argument (§2.3).
- Poseidon: serves the nullifier PRF (§2.7), where a keyed in-circuit PRF rather than a CRH is required. Unlike Sinsemilla, its security rests on cryptanalytic confidence in algebraic-attack resistance rather than on a hardness assumption such as DL.

The shift followed a consistent design principle: "transparent SNARK, recursion-ready" constituted the design target, and the designers engineered the Pasta cycle to fit.

The stack has since carried one correction and one split, neither touching its cryptography. The Action circuit exists in three versions — NU5's original, the NU6.2 emergency correction (ZIP-257, Final), and the NU6.3 Ironwood circuit with the cross-address input (§5.3) — named `InsecurePreNu6_2`, `FixedPostNu6_2`, and `PostNu6_3` in the released crate `orchard` 0.15.0 (`src/circuit.rs`; the `bef8a27` dev tree names the third variant `Ironwood`). And from NU6.3 the one protocol carries two value pools, the sealed Orchard pool and the current Ironwood pool (ZIP-229, Terminology; §5.1). The Halo 2 / Pasta stack is unchanged throughout.

4.4 Comparative summary

The proof size grew between Sapling and Orchard; this is the cost of transparency (a one-Action bundle's proof occupies $2720 + 2272 = 4992$ bytes in the v5/v6 encodings, including the $2 \log_2 2^{11} = 22$ group elements of the IPA rounds). The benefit, beyond the removal of trusted setup, is the foundation for recursion: future generations (Tachyon, eventual designs that aggregate many transactions under a single succinct proof) can build on Orchard's foundation without revisiting the cryptographic stack.

	Sprout	Sapling	Orchard
SNARK	BCTV14	Groth16	Halo 2
Curve	BN254	BLS12-381 + Jubjub	Pallas + Vesta
Trusted setup	Per-circuit (six-party MPC)	Per-circuit (large MPC)	None
Recursion-ready	No	No	Yes (via Pasta cycle)
Proof size	296 B	192 B	~5 kB (one Action)
Verification time	Constant (pairing)	Constant (3 pairings)	$O(\log n)$ amortised

Table 1: Three generations of Zcash shielded-payment proof systems. Orchard verification: $O(\log n)$ amortised ($n = 2^{11}$ rows; per-proof succinct checks, the linear MSM batched across proofs — the *Halo 2 Guide*). The Orchard column covers three circuit versions (the NU5 original; the NU6.2 correction, ZIP-257, Final; the NU6.3 Ironwood circuit with the cross-address input) and, from NU6.3, two value pools (§5.1).

5 The Ironwood pool

The NU6.3 network upgrade (ZIP-258, Draft) closes the story this volume has told so far and opens its successor: it seals the value pool that has carried Orchard payments since NU5 and starts a second pool — *Ironwood* — running the same cryptography. ZIP-229 (Draft) fixes the vocabulary this section uses throughout. The *Orchard protocol* is the shared design: the Pasta curves, Sinsemilla, the Action circuit (as modified by ZIP-2006), the note, commitment, nullifier, and key constructions, and the note encryption (as modified by ZIP-2005) — everything the preceding sections constructed. The *Orchard pool* and the *Ironwood pool* are two value pools of that one protocol, each with its own note commitment tree, anchor, and chain value pool balance. Nothing in the preceding sections is discarded; what changes is which pool new value may enter, and how an Ironwood note derives its trapdoor.

As in §6, claims about deployed behaviour cite the implementation and the specification — here the released crate `orchard` 0.15.0 (the version `librustzcash` pins; the Ironwood note-format, note-encryption, and v6-digest work ships in it), the `orchard` git tree at `bef8a27` (a 0.14.0 dev tree predating the release, cited for the circuit and cross-address work it carries), `librustzcash` at commit `e30517e4`, and `protocol.tex` and the ZIPs at `zips` commit `69610984` — and forward-looking claims carry one of the three epistemic bins *specified*, *designed but unspecified*, or *open problem*.

5.1 Two pools, one protocol: activation and framing

Activation. NU6.3 carries consensus branch identifier `0x37A5165B`. Its activation heights are fixed by the deployed validator (`zebra-chain/src/parameters/constants.rs`, with `librustzcash` hardcoding the same values in `zcash_protocol/src/consensus.rs`), while the ZIP-258 draft still lists Mainnet TBD — implementations ahead of their specification, which we flag rather than resolve. From activation, three consensus rules seal the Orchard pool to *spend-only*, and they bind every transaction regardless of version:

- a coinbase transaction must carry an empty Orchard component;
- every Orchard-pool Action must have cross-address transfers disabled ($\mathsf{enableCrossAddress} = 0$, §5.3) — enforced by the Action-circuit *verifying key*, which consensus selects by branch, so a v5 transaction cannot bypass the rule;
- no new value may enter the pool: $v_{\text{balance}}^{\text{Orchard}} \geq 0$ per transaction (§5.4).

All three are ZIP-258 rules; ZIP-229 states the second's version-independence. Notably, `zcashd` is not expected to implement NU6.3 at all; `zebra` is expected to be the only full-validator implementation from activation onward (ZIP-258).

Keys are protocol-scoped. Orchard spending-key and viewing-key material grants authority to spend or view notes in *both* pools; a receiver, and the corresponding ivk, is scoped to the Orchard protocol, not to a pool (ZIP-229; ZIP-326). The entire key ladder of §2.4 therefore survives unchanged: an existing wallet's keys and addresses work in Ironwood as they stand. What distinguishes the pools is state, not identity: separate, independent note commitment trees and nullifier sets (ZIP-229), separate anchors, and separate chain value pool balances. The Ironwood tree starts empty at activation and uses the identical depth-32 $\mathsf{MerkleCRH}^{\mathsf{Orchard}}$ construction of §2.6 (capacity 2^{32}, ZIP-258); the Orchard tree keeps growing after activation — every Orchard-pool Action still appends a cmx, since spends under the cross-address restriction are paired with fabricated or dummy outputs (§5.3) — and its anchors remain live for spend proofs. Nullifier derivation itself (§2.7) is unchanged in both pools; a note's nullifier is checked only against its own pool's set.

The v6 transaction format. A v6 transaction (`V6_TX_VERSION = 6`, version group `0xD884B698`; `librustzcash` `zcash_protocol/src/constants.rs`) is the v5 format — with enableSpendsOrchard/enableOutputsOrchard renamed to enableSpends/enableOutputs and bit 2 of flagsOrchard now carrying enableCrossAddress — plus an Ironwood component that reuses the `OrchardAction` encoding byte-for-byte (820 bytes per Action; ZIP-229):

Field	Size (bytes)	Content
`nActionsIronwood`	`compactSize`	Action count n
`vActionsIronwood`	$820 \cdot n$	`OrchardAction` encoding
`flagsIronwood`	1	same bit layout as `flagsOrchard`
`valueBalanceIronwood`	8	net flow $v_{\text{balance}}^{\text{Ironwood}}$
`anchorIronwood`	32	a root of the *Ironwood* tree
`sizeProofsIronwood`	`compactSize`	length of `proofsIronwood`
`proofsIronwood`	$2720 + 2272 \cdot n$	aggregate Halo 2 proof
`vSpendAuthSigsIronwood`	$64 \cdot n$	one SpendAuthSig per Action
`bindingSigIronwood`	64	the pool's binding signature

Table 2: The Ironwood component of a v6 transaction (ZIP-229). The fields after the count are present iff $n > 0$. The proof size formula is the same $2720 + 2272n$ as the Orchard component (Table 1).

Value conservation runs per pool: the Ironwood component has its own balancing value $v_{\text{balance}}^{\text{Ironwood}}$ and its own binding signature, constructed exactly as in §2.9 over the Ironwood Actions' $\mathsf{cv}^{\mathsf{net}}$ values. ZIP-229 states the note-level situation exactly: every Ironwood-pool output note uses the quantum-recoverable note plaintext format (lead byte `0x03`, §5.2), no Orchard-pool output note does, and this is "the only note-level distinction between the two pools".

The v6 digest. The ZIP-244 tree of §2.9 gains an Ironwood branch,

$$\mathsf{sighash} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \mathsf{branchid},\ d_{\text{hdr}} \,\|\, d_{\text{trans}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}} \,\|\, d_{\text{Irw}}\big),$$

with new personalisations `ZTxIdIronwd_H_v6`, `ZTxIdIrnActCH_v6`, `ZTxIdIrnActMH_v6`, `ZTxIdIrnActNH_v6`, `ZTxAuthIrnwdH_v6` and revised v6 personalisations for the Sapling and Orchard branches (`ZTxIdSSpendNH_v6`, `ZTxAuthSapliH_v6`, `ZTxIdOrchardH_v6`, and `ZTxAuthOrchaH_v6`); an absent component hashes the empty string under its personalisation (ZIP-229). This digest tree is deployed: the released crate `orchard` 0.15.0 (the `librustzcash` pin) defines the seven Orchard and Ironwood `_v6` personalisations and selects them per pool and transaction version (`bundle/commitments.rs`, enum `BundleCommitmentFormat`), and `librustzcash` at `e30517e4` assembles the v6 digests in production: `digest_orchard` and `digest_ironwood` in `zcash_primitives transaction/txid.rs` call `Bundle::commitment`, the file itself defines only the two Sapling personalisations `ZTxIdSSpendNH_v6` and `ZTxAuthSapliH_v6` (lines 44 and 54), and `transaction/tests.rs` re-derives the empty-component digests as test vectors. The

`bef8a27` dev tree predates this work: there `bundle/commitments.rs` (`:7--11`) defines only the five non-versioned personalisations (`ZTxIdOrchardHash`, `ZTxIdOrcActCHash`, `ZTxIdOrcActMHash`, `ZTxIdOrcActNHash`, `ZTxAuthOrchaHash`), with no per-pool or per-version dispatch. One structural change rides along: v6 moves the Sapling, Orchard, and Ironwood *anchors* from effecting data (under the txid) to authorizing data (under the auth digest, committed only when the component spends). A transaction can then be re-anchored — its membership proofs rebased onto a later root — without changing its txid, a design ZIP-229 credits to Penumbra.

One circuit, one verifying key. Both pools share the post-NU6.3 Action circuit and its keys. In the released `orchard` 0.15.0 a `BundleVersion` pairs a `ValuePool` (`Orchard` or `Ironwood`) with a `ProtocolVersion` (`InsecureV1`, `V2`, or `V3`): `circuit_version()` maps `V3` to `OrchardCircuitVersion::PostNu6_3` for both pools, `note_version()` maps `Orchard` to `V2` and `Ironwood` to `V3`, and `permits_cross_address_transfers()` is false exactly for (`V3`, `Orchard`) (`src/lib.rs`, `src/bundle.rs`). The in-circuit gate is `supports_cross_address_restriction()` on `OrchardCircuitVersion`, true only for `PostNu6_3` (`src/circuit.rs`); the `bef8a27` dev tree's `BundleFormat` enum and `Ironwood` circuit-version name were reworked into these before release. The pools are distinguished by their trees, nullifier sets, balances, and component position — not by separate circuits.

5.2 The Ironwood note: lead byte 0x03 and the hashed trapdoor

An Ironwood note is the same tuple $\mathbf{n} = (\mathsf{addr}, v, \rho, \psi, \mathsf{rcm})$ as Definition 2.5, committed by the same NoteCommit of §2.5; ZIP-2005 carries the pool distinction on the note *plaintext*, through its lead byte. What changes is that lead byte and, through it, the derivation of rcm (ZIP-2005, Proposed; activation bound to NU6.3). The format is specified — ZIP-2005 § 4.7.3, landed in `protocol.tex` at `zips` commit `69610984` — and deployed in the released `orchard` 0.15.0 (the `librustzcash` pin): enum `NoteVersion` (`V2`/`V3`) carries the lead byte, `Note::from_parts` takes it as a fifth argument, and `Note::rcm()` dispatches on it (`src/note.rs`). The `bef8a27` dev tree predates the release: its `Note::from_parts` (`src/note.rs:182`) still takes the four components (recipient, v, ρ, rseed) with no version tag, and its `Note::rcm()` is the legacy single-tag trapdoor of Remark 5.3.

Definition 5.1 (Allowed lead bytes, pool-parameterised)**.** Let pool be the chain value pool of a note received at block height h — one of SaplingPool, OrchardPool, or IronwoodPool — and let h_{Canopy} be the activation height of the Canopy network upgrade, which deployed ZIP-212's rseed-derived note plaintext format and its `0x02` lead byte. Then

$$\mathsf{allowedLeadBytes}^{\mathsf{pool}}(h) := \begin{cases} \{\texttt{0x01}\}, & h < h_{\mathsf{Canopy}}, \\ \{\texttt{0x01}, \texttt{0x02}\}, & h_{\mathsf{Canopy}} \le h < h_{\mathsf{Canopy}} + \mathsf{ZIP212GracePeriod}, \\ \{\texttt{0x02}\}, & \text{afterwards, if pool} \neq \mathsf{IronwoodPool}, \\ \{\texttt{0x03}\}, & \text{otherwise.} \end{cases}$$

For Ironwood-pool note plaintexts the only allowed lead byte is `0x03` (ZIP-2005 § 3.2.1 change; landed in `protocol.tex` at `zips` commit `69610984`). Ironwood coinbase outputs must carry it as well, the ZIP-213 analogue of the Sapling/Orchard `0x02` rule (ZIP-258).

Definition 5.2 (Ironwood note derivations)**.** For a lead-byte-`0x03` note with seed rseed and $\rho :=$

> $\mathsf{nf}_{\mathsf{old}}$, the sender derives, *in this order*,
>
> $$\begin{aligned} \mathsf{esk} &:= \mathrm{ToScalar}\big(\mathrm{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x04} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathrm{Vesta}}}, \\ \psi &:= \mathrm{ToBase}\big(\mathrm{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x09} \,\|\, \rho)\big) \in \mathbb{F}_{p_{\mathrm{Pallas}}}, \\ \mathsf{rcm} &:= \mathrm{ToScalar}\big(\mathrm{PRFexpand}_{\mathsf{rseed}}(\mathsf{pre\_rcm})\big) \in \mathbb{F}_{p_{\mathrm{Vesta}}}, \\ \mathsf{pre\_rcm} &:= \mathtt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi), \end{aligned}$$
>
> each field in its canonical byte encoding: the points star-encoded (32 bytes each), v as 8 and ρ, ψ as 32 little-endian bytes, so pre_rcm is a 137-byte string (ZIP-2005 § 4.7.3 change; `protocol.tex` at `zips` commit `69610984`, where the hash is named $\mathsf{H}^{\mathsf{rcm,Orchard}}_{\mathsf{rseed}}$).

The esk and ψ derivations are byte-identical to §2.8; only rcm changes, and with it the order — rcm now comes last because its input includes ψ (ZIP-2005). The same $\mathsf{g_d}^{\star}$, $\mathsf{pk_d}^{\star}$ encodings feed both pre_rcm and the NoteCommit input of §2.5. The purpose is post-quantum *binding*: every note field now traverses BLAKE2b on the way to the trapdoor, so an adversary constrained to treat PRFexpand as a random oracle cannot vary any field without changing rcm (§6.4, which owns the security analysis, the recovery statement, and ZIP-2005's specified-but-undeployed qsk/qk key branch). Every Ironwood-pool output note is a recoverable note; no Sapling or Orchard-pool output note is (ZIP-2005).

Remark 5.3 (The legacy derivation, historically)**.** Lead-byte-`0x02` notes — all Orchard-pool notes, before and after NU6.3 — derive $\mathsf{rcm} := \mathrm{ToScalar}(\mathrm{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x05} \,\|\, \rho))$ as in §2.8. The specification packages both under one dispatcher, $\mathrm{Derive\_rcm}^{\mathsf{Orchard}}_{\mathsf{rseed}}(\mathsf{leadByte}, \ldots)$, selecting $\mathtt{0x05} \,\|\, \rho$ for `0x02` and the hashed pre_rcm for `0x03` (ZIP-2005 § 4.7.3 change). The dev-tree `Note::rcm()` at `bef8a27` implements only the legacy branch above (`src/note.rs:132--136`), with `PrfExpand::ORCHARD_RCM = 0x05` (`zcash_spec` 0.2.1, `src/prf_expand.rs:88`), no version dispatch, and no `0x03`/hashed-pre_rcm branch. The released `orchard` 0.15.0 that `librustzcash` pins adds the `NoteVersion` dispatch — `V2` to `rcm_v2`, `V3` to `rcm_v3` over the `0x0B`-separated pre_rcm (`src/note.rs`) — deploying the derivation of Definition 5.2.

Decryption is reordered. Trial decryption (§2.8) rederives the note's quantities to check the commitment, and the lead-byte-`0x03` rcm depends on $\mathsf{g_d}, \mathsf{pk_d}$, and ψ; the specification therefore reorders both the ivk and fvk decryption procedures to recover $\mathsf{g_d}$ and $\mathsf{pk_d}$ first, then ψ, then rcm (ZIP-2005 §§ 4.20.2–4.20.3 changes). The routing is deployed: the released `orchard` 0.15.0 routes the two lead bytes to two domains, `OrchardDomain` (`0x02`) and `IronwoodDomain` (`0x03`) (`src/note_encryption.rs`), which `zcash_client_backend` scanning consumes (`src/scanning.rs`); the `bef8a27` dev tree still has a single `0x02`-only `OrchardDomain`. Because keys are protocol-scoped (§5.1), the *same* ivk trial-decrypts both pools — one key, two domains (ZIP-326; `zcash_client_backend src/scanning.rs`). The light-client surface follows suit: compact blocks gain an `ironwoodActions` list (the same `CompactOrchardAction` message) and an `ironwoodCommitmentTreeSize`, and the synchronisation of §2.8 runs per pool against its own tree (`zcash_client_backend proto/compact_formats.proto`).

5.3 The cross-address restriction in the Orchard pool

The rule that seals the Orchard pool most deeply is in-circuit: from NU6.3, every Orchard-pool Action must have cross-address transfers disabled, so that value inside the pool can no longer move between addresses — an Orchard-pool spend can pay only its own receiver. The motivation is the recoverability guarantee of §6.4: legacy `0x02` notes are unrecoverable, so consensus stops their circulation and funnels value out of the pool, while still letting owners spend (§5.4).

Definition 5.4 (Action statement, the A10 constraint in full)**.** The tenth public input of Definition 2.9, disableCrossAddress, sits at instance index 9 (after anchor, the two $\mathsf{cv}^{\mathsf{net}}$ coordinates, $\mathsf{nf}_{\mathsf{old}}$, the two rk coordinates, cmx, enableSpends, enableOutputs), and constraint A10 of the relation is, writing $\delta :=$ disableCrossAddress:

[A10] Cross-address gating — when $\delta \neq 0$, the new note's receiver equals the spent note's:

$$\delta \cdot \big(x(\mathsf{g}_{\mathsf{d}_{\mathsf{old}}}) - x(\mathsf{g}_{\mathsf{d}_{\mathsf{new}}})\big) = 0, \qquad \delta \cdot \big(y(\mathsf{g}_{\mathsf{d}_{\mathsf{old}}}) - y(\mathsf{g}_{\mathsf{d}_{\mathsf{new}}})\big) = 0,$$
$$\delta \cdot \big(x(\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}}) - x(\mathsf{pk}_{\mathsf{d}_{\mathsf{new}}})\big) = 0, \qquad \delta \cdot \big(y(\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}}) - y(\mathsf{pk}_{\mathsf{d}_{\mathsf{new}}})\big) = 0,$$

four coordinate equalities forcing $\mathsf{g}_{\mathsf{d}_{\mathsf{old}}} = \mathsf{g}_{\mathsf{d}_{\mathsf{new}}}$ and $\mathsf{pk}_{\mathsf{d}_{\mathsf{old}}} = \mathsf{pk}_{\mathsf{d}_{\mathsf{new}}}$ as full curve points — equality of receivers, not merely of x-coordinates. The constraint shape is the product gate A3 already uses ($v_{\mathsf{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$), reused on four extra rows of the same `q_orchard` gate (crate `orchard` at `bef8a27` `src/circuit.rs`: instance offsets `ANCHOR` through `DISABLE_CROSS_ADDRESS`, rows assigned in `synthesize_cross_address_checks`; the historical nine-element instance encoding is commitment-identical when $\delta = 0$, so pre-NU6.3 proofs and keys are untouched).

Wire encoding and polarity. The flag travels as bit 2 of the component's flags byte, in the *enabled* sense (enableCrossAddress; bits are spends `0b001`, outputs `0b010`, cross-address `0b100` — `librustzcash pczt/src/orchard.rs` carries both canonical values, `0b0000_0011` for Orchard and `0b0000_0111` for Ironwood). The polarity is backward-compatible by design: bit $2 = 0$ — the restricted state consensus requires for post-NU6.3 Orchard-pool spends — is exactly what v5 signers treating the bit as reserved-zero already produce, so existing hardware signers (ZIP-229 names the Keystone) keep signing v5 Orchard unshielding transactions without a firmware update. The Ironwood component carries the same flag and sets it: cross-address transfers stay enabled there.

Spending under the restriction. An Action spends one note and creates one (§2.11); if the created note must pay the spent note's own receiver, how does an owner pay anyone? By exiting the pool: the real value leaves through $v^{\mathsf{Orchard}}_{\mathsf{balance}} > 0$ (§5.4), and the Action's output slot is filled with a *fabricated* zero-valued note to the spent note's own receiver. The wallet must fill that output's C_{enc} with *random* bytes: a real encryption would let any ivk holder — including a quantum adversary who recovered ivk from a published address, the very adversary of §6.1 — decrypt it and link the Action's nullifier to the address (ZIP-326). In a PCZT (a *partially created Zcash transaction*, the format that carries an in-progress transaction between its constructor, prover, and signers) the fabricated output carries its recipient, value, and rseed explicitly, so a signer can classify it as a tolerable dummy rather than an opaque payment (ZIP-326; `librustzcash pczt/src/orchard.rs`). One gap remains open by construction: paying *into* an Orchard-pool receiver post-NU6.3 requires a spend at that same receiver, hence the spending key, so consensus cannot fully prevent self-sends; ZIP-326 wallet rules close the gap by forbidding sends to external Orchard-pool receivers.

Status of ZIP-2006. The restriction is deployed and enforced by consensus, but its owning ZIP does not yet exist as text: `zip-2006` is a nine-line Reserved stub ("Restricting Transfers into the Orchard Pool"; owner Daira-Emma Hopwood; created 2025-06-20; discussion `zcash/zips#1305`). *Specified* (in ZIP-229/258): the flag encoding, its consensus rule, and the verifying-key selection by branch. *Designed but unspecified*: the normative semantics — ZIP-326 phrases the restriction as equality of "expanded receivers" and carries a `[TODO define]` in its own text; the enableCrossAddress → disableCrossAddress instance mapping and the dummy-spend treatment are likewise reconstructable only from ZIP-229/258/326 and crate `orchard` at `bef8a27`, as Definition 5.4 does.

5.4 Migration and the turnstile

Value leaves the Orchard pool through a *turnstile*: a one-way, publicly metered gate, the same instrument Zcash used for the Sprout retirement. The consensus content is two rules already stated. Per transaction, from activation,

$$v_{\text{balance}}^{\text{Orchard}} \geq 0 \qquad \text{(any transaction version; ZIP-258)},$$

so value may exit but never enter; and the chain maintains an Ironwood analogue of every pool-balance rule of §2.12 (the ZIP-209 extension):

$$\mathsf{IronwoodPoolBalance} := -\sum_{\text{tx}} v_{\text{balance}}^{\text{Ironwood}}(\text{tx}),$$

zero before NU6.3, never negative, with the sum of all pool balances bounded by MAX_MONEY (ZIP-258). A migrating transaction spends Orchard-pool notes with $v_{\text{balance}}^{\text{Orchard}} > 0$ — releasing value to the transaction's transparent value pool — and absorbs it into an Ironwood component with $v_{\text{balance}}^{\text{Ironwood}} < 0$. Both fields are public: the turnstile reveals the amounts crossing between pools in each transaction, exactly as ZIP-229's privacy analysis states, and no per-transaction or aggregate cap is specified anywhere. The remaining Ironwood consensus rules are the Orchard ones of this volume, re-instantiated: anchor$_{\text{Ironwood}}$ must be an earlier block's final Ironwood treestate (§2.6), the tree capacity is 2^{32}, and the ZIP-221 history tree (the Merkle mountain range over the chain's history whose root every block header commits to; *FlyClient Guide*, §"NU5: the commitment becomes one of two") gains three Ironwood fields (earliest root, latest root, transaction count) aggregated exactly like the Orchard trio (ZIP-258).

Wallet posture. Wallets SHOULD proactively move *all* funds — transparent, Sprout, and Sapling included — into the Ironwood pool, the only pool whose output notes are recoverable, and SHOULD treat the sweep as ongoing (ZIP-2005; §6.4). The deployed change-routing logic implements the turnstile's grain: change from Ironwood value stays in Ironwood, and after activation change may return to the Orchard pool only when the transaction spends Orchard notes *and* strictly less value returns than the spends remove; otherwise change is diverted to Ironwood (function `select_change_pool`, `librustzcash zcash_client_backend/src/fees/common.rs`). Scanning narrows correspondingly: ZIP-326's scan ranges close the Orchard pool off — external receivers scan from the birthday only to activation, internal ones until the wallet observes a zero Orchard balance after activation, a watermark that is sound precisely because $v_{\text{balance}}^{\text{Orchard}} \geq 0$ means a zero balance can never become nonzero — while Ironwood scans from the later of birthday and activation to the tip (ZIP-326).

Status of ZIP-318. The migration ZIP itself — "Orchard to Ironwood Migration" — is an eleven-line Reserved stub (owners Schell Carl Scivally and Pacu Gindre; created 2026-06-16; discussion `zcash/zips#1315`). Everything consensus-level above is specified in ZIP-229/258; what ZIP-318 is expected to own — batching, scheduling, and amount-disclosure guidance in the style of the Sprout-to-Sapling migration (ZIP-308), mitigating the public crossing amounts — exists nowhere as text and is an *open problem*.

6 Post-quantum security of the Orchard protocol

Every group-structured assumption in Theorem 3.5 — DL on both Pasta curves and, for property (iv), DDH on Pallas — falls to Shor's algorithm in polynomial time on a cryptographically relevant quantum computer (CRQC; the *Crypto Guide*, §"The quantum caveat: Shor's algorithm"); the theorem's remaining assumptions, the PRF security of Poseidon and the security of the KDF/AEAD, are symmetric and face only the generic speedups of §6.3. The productive question is therefore not *whether* the

assumptions fail — they all do, together — but *when the failure matters*. We organise this section by *reversibility*. A break is *retroactive* when data already recorded on chain becomes exploitable the day the adversary exists: no later consensus change can help, because the data is public and permanent. A break is *prospective* when it requires a validator to *accept* a forged object after the adversary exists: disabling the affected protocol before that day removes every future acceptance event and mitigates the break completely. The Orchard protocol — both its pools — has exactly one retroactive break (§6.1); everything else on its assumption surface is prospective (§6.2) or faces only generic quantum speedups (§6.3). This section is the Orchard core seen from inside; the same reversibility axis applied across every layer of Zcash — the transparent stack, proof of work, the frontier protocols, and the migration arithmetic — is the *PQ Guide*'s subject.

Much of this section concerns design-stage material, so we make the epistemic status of every forward-looking claim explicit, in one of three bins: *specified* (a published artifact pins the construction down, which we cite), *designed but unspecified* (an informal source describes the design; we say what a specification would still need to fix), and *open problem* (no artifact exists). Claims about the deployed protocol instead cite the implementation and the specification as elsewhere in this volume. Ground truth for this section: `protocol.tex` and the ZIPs at `zips` commit `69610984` (2026-07-09); crate `orchard` at git commit `bef8a27e` (v0.14.0, 2026-06-16) for circuit claims; and the released crate `orchard` 0.15.0 (the `librustzcash` pin) for the `0x03` note path; we verified each cited artifact at these pins.

6.1 The retroactive break: harvest now, decrypt later against note encryption

Each Action publishes the pair $(\mathsf{epk}, C_{\mathrm{enc}})$ (§2.8; crate `orchard`, `src/note_encryption.rs`). Confidentiality of C_{enc} rests on exactly one discrete-log-dependent link: the one-shot Diffie–Hellman on Pallas between esk and $\mathsf{pk_d}$ (protocol §5.4.5.5; the esk derivation is `src/note.rs`, the key agreement `src/keys.rs`). The KDF and the AEAD above it are symmetric and are not the weak layer (§6.3).

A CRQC that knows a recipient's address $(d, \mathsf{pk_d})$ computes

$$\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d}$$

by Shor — one discrete logarithm for the lifetime of the address — and then runs the recipient's own trial-decryption loop of §2.8 over the recorded chain: for every Action, $[\mathsf{ivk}]\,\mathsf{epk}$ recovers K_{enc}, and the AEAD tag identifies which ciphertexts belong to the address. The adversary obtains, for every note ever sent to that address, the plaintext $(d, v, \mathsf{rseed}, \mathsf{memo})$: every amount, every memo, the complete receiving history — and equally every *future* ciphertext to the same address. Two limits bound the damage. First, ivk is a viewing capability, not a spending one (§2.4): no spend authority is conferred, and ask is not exposed. Second, the attack is keyed by the address: without $(d, \mathsf{pk_d})$ the adversary has neither the base $\mathsf{g_d}$ nor the target $\mathsf{pk_d}$, and the recorded chain reveals nothing further even to an unbounded adversary (§6.2). ZIP-2005 records the same boundary: the harvest-now-decrypt-later exposure requires "both the ciphertext ... and the receiver address".

The break is retroactive by construction. The pair $(\mathsf{epk}, C_{\mathrm{enc}})$ is consensus data, replicated on every full node forever; its secrecy was fixed at recording time by one specific DH instance; any protocol upgrade alters only ciphertexts not yet created. An adversary needs no quantum computer today — only storage — and every Orchard output recorded since NU5 activation is already in that corpus, which accrues Ironwood outputs from NU6.3 activation onward (ZIP-2005, Non-requirements).

Open problem: mitigation. Nothing is deployed, and nothing is specified, that protects even future ciphertexts. ZIP-2005 (Proposed) excludes privacy by an explicit Non-requirement, states that the exposure persists for all pools including Ironwood, and says only that mitigations for future transfers are "under consideration". The tracking issues (`zcash/zips#1133`, open since 2022; `#1134`, open since 2016) carry discussion, not drafts; the `zips` repository at commit `69610984` contains no occurrence of ML-KEM, Kyber, ML-DSA, or a lattice scheme. A future hybrid key encapsulation would in any case protect future outputs only; the recorded ciphertexts are unrepairable.

6.2 Prospective breaks: the discrete-logarithm integrity stack

The remainder of the assumption surface protects *integrity*, and every piece of it fails against the same adversary. The reductions of §3 each run equally well in reverse: the assumption a CRQC falsifies is exactly the one the security proof consumes.

- *Sinsemilla binding.* Collision resistance reduces to the discrete-log relation problem among the GroupHash generators (Proposition 3.1). A CRQC computes every pairwise logarithm, after which each Sinsemilla commitment opens to arbitrary messages: a second note under an on-chain cmx (breaking balance through NoteCommit non-binding), a fabricated membership path under any anchor (§2.6), and alternative pairs $(\mathsf{ak}', \mathsf{nk}')$ opening the same $\mathsf{Commit}^{\mathsf{ivk}}$ — the Faerie-Gold-style Spendability attacks ZIP-2005 catalogues. A balance violation committed this way is bounded only by the ZIP-209 turnstile and need not be detected at all.
- *Value commitments and the binding signature.* Binding of $\mathsf{cv}^{\mathsf{net}}$ is the unknown discrete-log relation between V^{Orchard} and R^{Orchard} (Definition 2.8, Proposition 3.4); the binding signature is Schnorr under DL in the ROM. Knowledge of $\log_{V^{\mathsf{Orchard}}} R^{\mathsf{Orchard}}$ reopens any recorded or fresh cv to any value and yields bsk for any target bvk: silent inflation, again turnstile-bounded (crate `orchard`, `src/value.rs`).
- *RedPallas spend authorisation.* Unforgeability is DL in the ROM (Proposition 3.3). One logarithm — of ak, or of any published rk — forges spend authorisation at will (crate `orchard`, `src/primitives/redpallas.rs`); combined with the next item it completes arbitrary theft.
- *IPA knowledge soundness.* The Action SNARK is sound under DL on Vesta with Fiat–Shamir in the ROM (§3.7). A CRQC forges accepting proofs of false Action statements: minting from nothing, spending any commitment on chain, indistinguishable from honest proofs. This is the single most concentrated failure point — every other item's exploitation route runs through it (crate `halo2_proofs`, `src/poly/commitment.rs`).

Why forking in time is a complete mitigation. Each attack above must place a forged object — a proof, a signature, a commitment opening — before a validator and have it *accepted*, and acceptance happens at current height under current consensus rules. Disabling the DL-dependent pools before a CRQC exists removes every future acceptance event; transactions already recorded were validated when the assumptions held and are never re-adjudicated. The breaks are therefore fully prospective, with two residues. Switch-off must precede the first forgery — an attack landed in the exposure window is not repaired afterwards (§6.4) — and honest funds inside a disabled pool need some other way out, which is precisely the problem ZIP-2005 exists to solve.

What the recorded chain does not leak. The same recorded data that becomes forgeable leaks no secrets, quantumly or otherwise. The value commitment is perfectly hiding, so recorded cv values reveal nothing about amounts even to an unbounded adversary (§2.9). The randomiser α is uniform, so recorded rk values and signatures carry information-theoretically no linkage to ak or ask (§3.4). Proof transcripts are statistically simulatable without the witness (§3.6). Nullifiers are keyed by nk, which never appears on chain (§2.7). Consequently a quantum adversary who does not know a recipient's address learns nothing from the recorded chain — ZIP-2005 reaches the same conclusion — and the sole retroactive channel is the address-keyed decryption of §6.1.

6.3 Primitives facing only generic speedups

Two primitive families in Orchard carry no discrete-log structure. BLAKE2b serves $\mathsf{PRFexpand}$ and with it every key and note-randomness derivation (§2.4, §2.8), the KDF and the outgoing cipher key, the RedPallas challenge hash, and the ZIP-244 sighash; the ZIP-32 Orchard key tree is hardened-only,

built entirely from these derivations, with no elliptic-curve operation anywhere on the derivation path (ZIP-32, Final; crate `orchard`, `src/zip32.rs`). Poseidon serves the nullifier PRF, keyed by nk (§2.7; `src/note/nullifier.rs`).

Against these, a quantum adversary gets generic speedups and nothing more — with the usual square-root caveat stated exactly. Grover search halves effective preimage security (the *Crypto Guide*, §"The quantum caveat: Shor's algorithm"): 2^{256} classical evaluations become $\sim 2^{128}$ quantum ones, a margin the parameters absorb. For collisions the picture is better still: the Brassard–Høyer–Tapp $2^{n/3}$ algorithm requires random access to a quantum memory of $\sim 2^{92}$ bits at these output sizes, and the best *known* implementable generic quantum collision attack remains the classical parallel one — the assessment ZIP-2005 adopts, flagged there as best-known rather than provably best. Poseidon adds one honest caveat of its own: its PRF security is a cryptanalytic heuristic with no standard-problem reduction, classically and quantumly alike (§4); a quantum adversary gains no identified structural advantage, and since nk stays off chain, recorded nullifiers remain unlinkable either way.

The consequence is the load-bearing fact of the next subsection: the symmetric backbone survives a discrete-log break intact. A seed holder can still prove, by re-executing derivations, that their keys descend from their seed — ownership remains *provable* after the assumption that made it *enforceable* has fallen.

6.4 ZIP-2005: quantum recoverability of Ironwood notes

ZIP-2005 ("Ironwood Quantum Recoverability", Proposed; created 2025-03-31) is the deployed-track response to the prospective breaks of §6.2. It does not make Orchard quantum-secure and says so; it arranges that funds survive the eventual switch-off.

Deployment. Activation is bound to the NU6.3 network upgrade (ZIP-258, Draft: consensus branch `0x37A5165B`); the scheduling follows the NU6.2 emergency circuit correction (ZIP-257, Final), which is why the originally planned earlier deployment slipped. The pool split itself is the subject of §5; what this subsection relies on is the seal. From activation the legacy Orchard pool is spend-only: a coinbase transaction must carry an empty Orchard component, every Orchard-pool Action must have $\mathsf{enableCrossAddress} = 0$, and no new value may enter the pool ($v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$) — the second rule enforced by the branch-selected Action-circuit verifying key, the other two as ordinary consensus rules (ZIP-258; ZIP-229).

Mechanism. Ironwood note plaintexts carry lead byte `0x03`, and with it the hashed trapdoor derivation of §2.8: rcm derives from the `0x0B`-separated $\mathsf{pre\_rcm}$ concatenating every note field (Definition 5.2), in place of the legacy $\mathtt{0x05} \,\|\, \rho$ input. Every note field now traverses BLAKE2b: an adversary constrained to treat $\mathsf{PRFexpand}$ as a random oracle cannot vary any field without changing rcm, so the composite of derivation and $\mathsf{NoteCommit}$ is post-quantum binding *provided a verifier checks the derivation* — $\mathsf{NoteCommit}$ itself, and the deployed Action circuit, are unchanged, and the binding claim is conditional on that future check. The same rederivation pattern covers $\mathsf{Commit}^{\mathsf{ivk}}$: the randomness $\mathsf{r}_{\mathsf{ivk}}$ descends from sk through $\mathsf{PRFexpand}$, or — for FROST-generated keys, where ask arises from distributed shares rather than from sk, and for hardware-held keys, where proof authority must remain separate from the on-device spend key — from a quantum key qk derived by one BLAKE3 `derive_key` compression from a secret qsk, with signatures of knowledge over the transaction sighash tying the claimed keys to the spend (the ZIP types it only as a message hash and pins no sighash algorithm). The non-normative Recovery Statement then proves, in a post-quantum knowledge-sound system over a re-hashed commitment tree: the key derivations ($\mathsf{BindKeys}$ and the $\mathsf{r}_{\mathsf{ivk}}$ branch), $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}$, $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, the ψ and rcm derivations, the commitment, a membership path, the nullifier, and the α-re-randomisation tying the public rk to ak — costed at 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, and a Merkle path, the BLAKE2b compressions dominating.

Security bounds as published. The quantum analogue of the collision resistance these arguments need is Unruh's *collapsing* property; Fehr's theorem for HAIFA-mode hashes reduces collapsing of the rcm derivation to modelling BLAKE2b's compression function as a random oracle — the same modelling the classical proofs already use. Both classical reductions (key binding and Spendability) are straight-line and never reprogram the oracle, so they lift cleanly to the quantum random-oracle model; by Zhandry's compressed-oracle technique the classical $O(q^2/N)$ collision bound becomes $O(q^3/N)$, giving

$$\varepsilon_{\mathsf{kb}}^{\mathsf{QROM}} \leq O(q^3/N), \qquad \varepsilon_{\mathsf{Spendability}}^{\mathsf{QROM}} \leq O(q^3/N) + \varepsilon_{\mathsf{kb}}^{\mathsf{QROM}},$$

with $N \approx 2^{254}$ the oracle output-space size (the Pallas scalar order, p_{Vesta} in this volume's notation). At $q \ll 2^{60}$ the worst-case bound is $\sim 2^{-74}$; the practically achievable bound stays near the classical $q^2/N \approx 2^{-134}$, since saturating the cubic bound needs the memory-infeasible algorithm of §6.3.

Scope: binding only, Orchard only. The ZIP repairs binding, not privacy: the harvest-now-decrypt-later exposure of §6.1 persists unchanged for every pool, Ironwood included. Sapling is deliberately excluded — "to reduce overall protocol complexity and analysis effort" — since the ZIP-32 Sapling derivations differ significantly from Orchard's hardened-only tree and would require their own analysis. The consequence is forced migration: Sapling, Sprout, and pre-Ironwood Orchard (0x02) notes are unrecoverable, hence permanently unspendable once their protocols switch off; wallets are directed to move all funds into Ironwood notes and to treat the sweep as an ongoing process, since non-recoverable notes may arrive at any time.

Specified. The 0x03 note derivation, the qsk/qk key path, and the consensus rules sealing the Orchard pool are pinned at ZIP rigor — ZIP-2005 (Proposed), ZIP-229 and ZIP-258 (Draft), all at `zips` commit 69610984 — and are enforced from NU6.3 activation: every Ironwood output is a recoverable note.

Designed but unspecified. The Recovery Protocol itself. ZIP-2005 presents the Recovery Statement in outline and states that its details are "subject to change"; a specification would need to pin down the post-quantum proof system (a stated requirement is that none be assumed now), the collapsing hash for the re-hashed commitment tree, the linking commitments between circuits, and the concrete signature-of-knowledge instantiations.

Open problem. A post-quantum proof system and signature scheme for *ongoing* spends — as opposed to one-shot recovery — exist nowhere as drafts (`zcash/zips#1134` carries discussion only). So does the schedule: every recoverability guarantee is conditional on legacy switch-off preceding the first quantum forgery, and attacks landed in the exposure window — Spendability roadblocks that permanently block recovery, spend-authority theft, turnstile-bounded inflation — are not repaired retroactively.

6.5 Strategy: recoverability, not resistance

The successor proving stack points the same way. Ragu, the recursive proof system under development for project Tachyon, is deliberately ECDLP-based over the same Pasta cycle, its authors citing "reduced concern (in the medium term)" for post-quantum soundness risks (`ragu-tachyon` book, `src/protocol/index.md`, commit 830bbcda, 2026-07-05). Read together with ZIP-2005, this fixes Zcash's quantum strategy precisely: *recoverability, not resistance* — keep discrete-log cryptography while it stands, because its integrity failures are prospective and fork-mitigable (§6.2), and pre-position notes so that funds provably survive the eventual transition (§6.4); the one cost this strategy cannot answer is the retroactive privacy channel of §6.1, which accrues damage now. Public claims that Zcash will be "fully post-quantum" on a twelve-to-eighteen-month horizon, with ML-KEM and ML-DSA

under test (CoinDesk, 2026-05-08), correspond to no ZIP, draft, or repository artifact, and ZIP-2005 itself disclaims making the protocol secure against quantum adversaries.